

高性能计算导论

第6讲：CUDA 编程模型-1

翟季冬
计算机系

目录

- GPU 背景
- CUDA 编程模型
- CUDA 规范与 API
- CUDA 例子
- CUDA 函数计时和调试
- 多 GPU 编程与动态并行
- GPU 编程生态

GPU 背景

GPU 图形处理单元 - Graphics Processing Unit

- GPU 是为了**图像处理和显示**的专用芯片
 - SIMD 体系结构
 - 1. **核心数量多**：GeForce RTX 3080 有 **8704** 个核心、频率：1.4GHz，但单核心计算能力远弱于 CPU 核心 Intel Xeon 8280 **28核心 2.7GHz**
 - 2. **整体算力强大**：GeForce 3080 在 Float32 上达到 **29.7 TFLOPS vs. Intel Xeon 8280 2.4 TFLOPS**
 - 3. **高速片上内存**：GeForce 3080 显存带宽 **760 GB/s vs. 8280 131.1GB/s**
- 广泛应用在计算图形学中
 - 图像渲染，光线追踪，3D建模等
- 两个主要 GPU 厂商：Nvidia 和 AMD



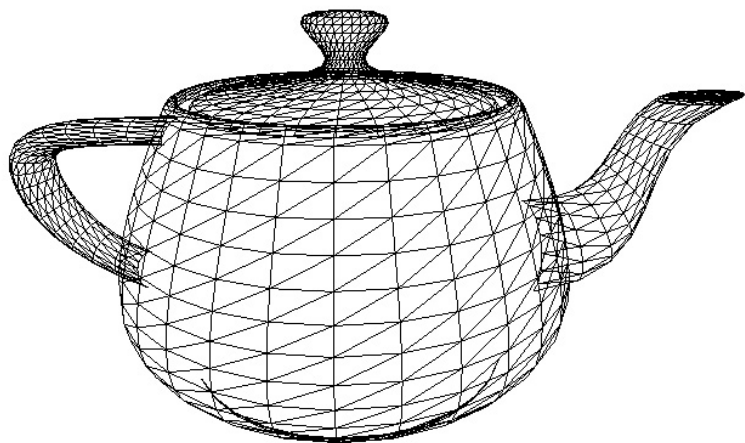
Nvidia 3080



Nvidia A100

GPU 背景

- GPU 原本用途：
 - 三维渲染



输入： 场景描述：
表面几何描述、表面材料、灯光等

输出： 场景的图片

GPU 背景

- 实时渲染 3 维复杂场景



孤岛惊魂 5

GPU 背景

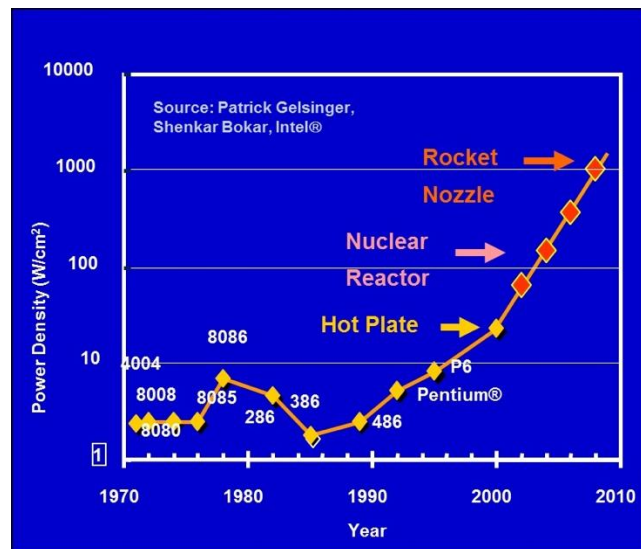
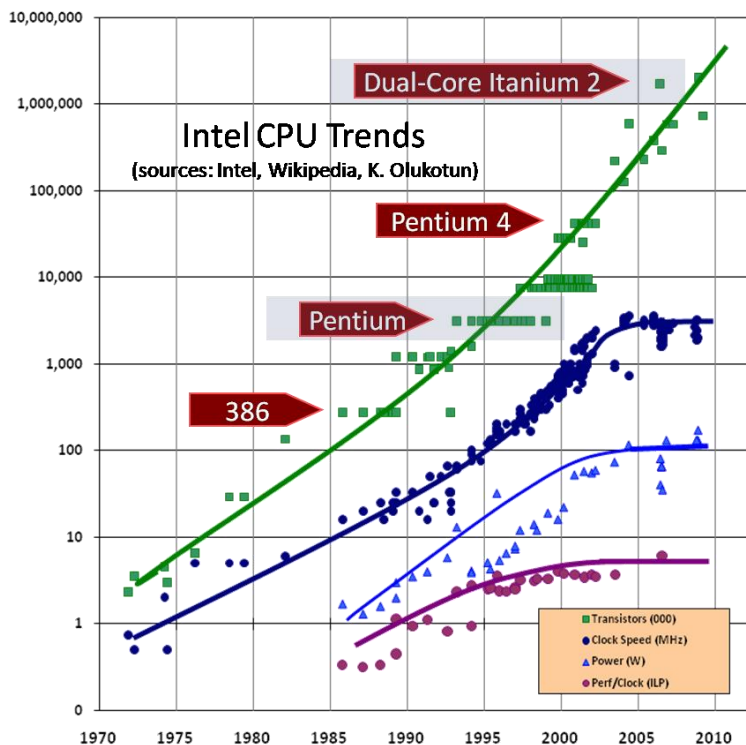
- 实时渲染 3 维复杂场景



Epic Games 2015

传统 CPU 扩展遇到瓶颈

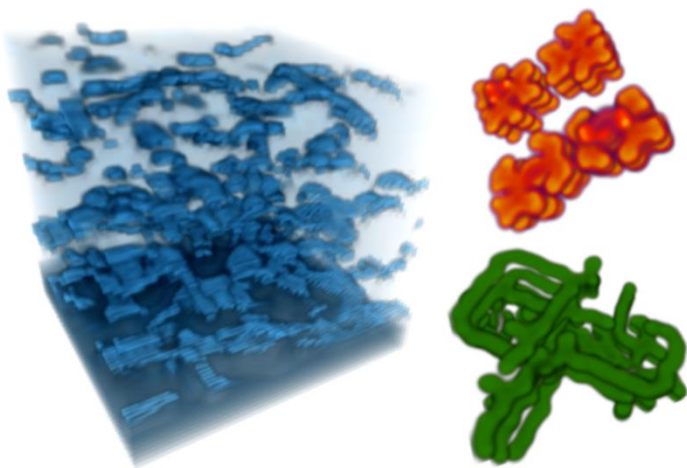
- 晶体管密度的提升 $\neq$ 性能的提升
 - 芯片功耗上升严重，时钟频率进展缓慢
 - 传统通用 CPU 性能发展遇到极大瓶颈



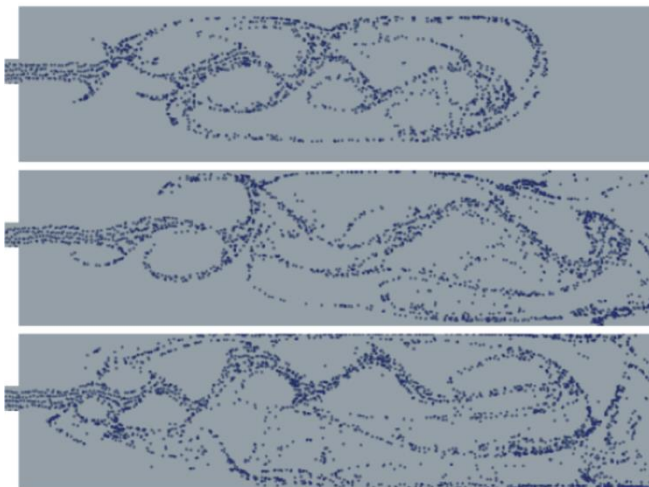
主频受功耗限制难以继续提升

GPGPU (General-Purpose Graphic Processing Unit)

- 探索 GPU 在通用计算上能力（2002年 - 2003年）
 - 通过挖掘数据并行来加速并行任务和数值计算
 - 科学或工程领域在 GPU 硬件上达到 100 倍或更高加速比
- 如何使用 GPU 进行通用计算？
 - 早期 GPGPU：
 - 使用 OpenGL 或 DirectX 接口进行一些通用计算
 - 后期 GPGPU：
 - 提供 CUDA 和 OpenCL 进行并行编程（2007年）



耦合映射格模拟 2002年



稀疏矩阵运算求解器 2003年

多核、并行与异构

单核时代

摩尔定律
晶体管密度提升

限制:
计算性能
复杂度

汇编 → C/C++ → Java ...



多核时代

摩尔定律
SMP

限制:
功耗
并行软件
可扩展性

Pthread → OpenMP ...



异构计算时代

摩尔定律失效
海量数据
异构加速器

限制:
编程模型
访存开销

Shader → CUDA ...



分布式计算时代

网络发展
大数据

限制:
同步开销
通信开销

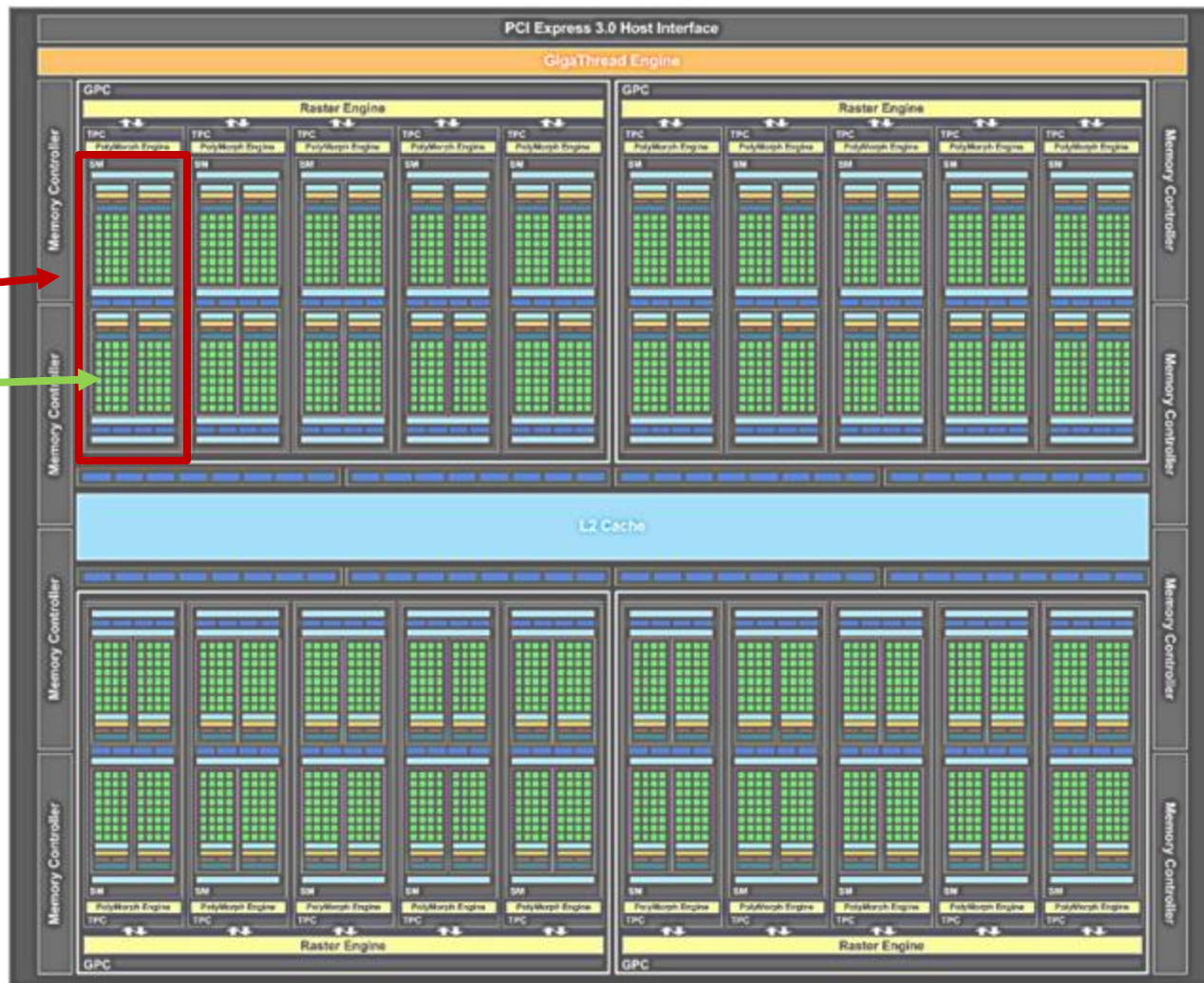
MPI → MapReduce ...

CUDA 编程模型

GPU 硬件组织

■ GPU 计算硬件组织:

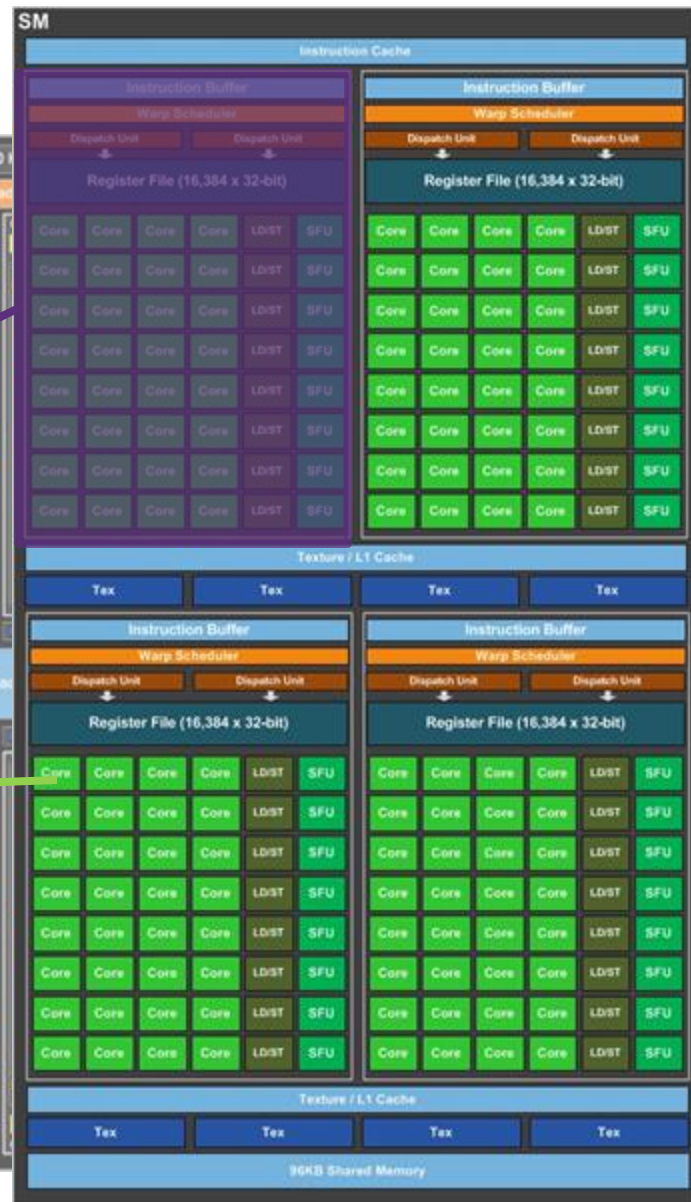
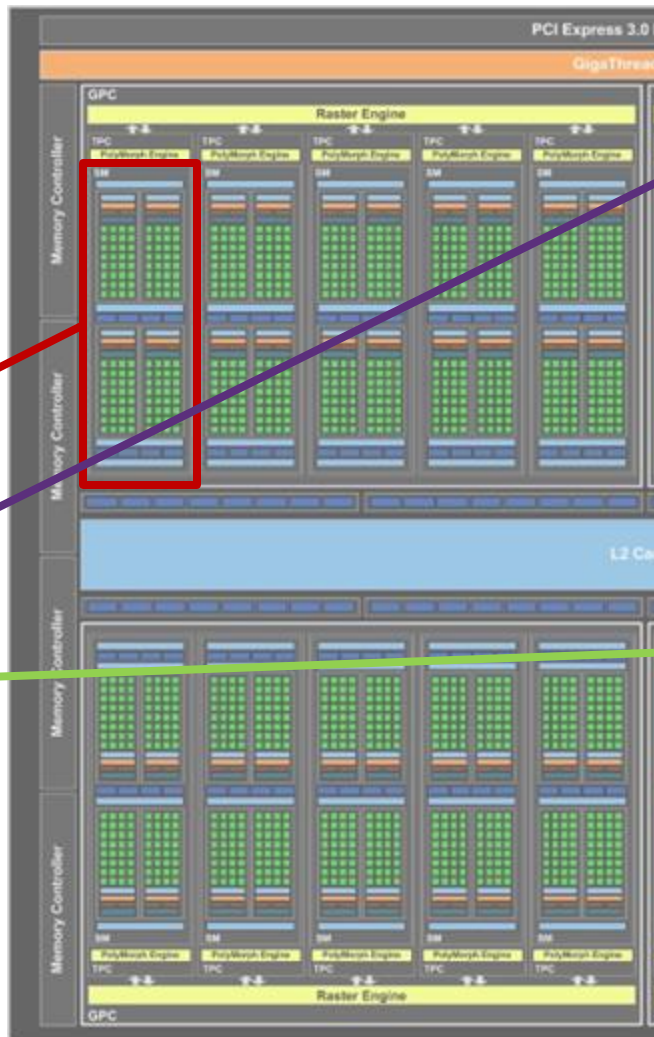
- GPU
- Streaming Multiprocessor (SM)
- Core



GPU 硬件组织

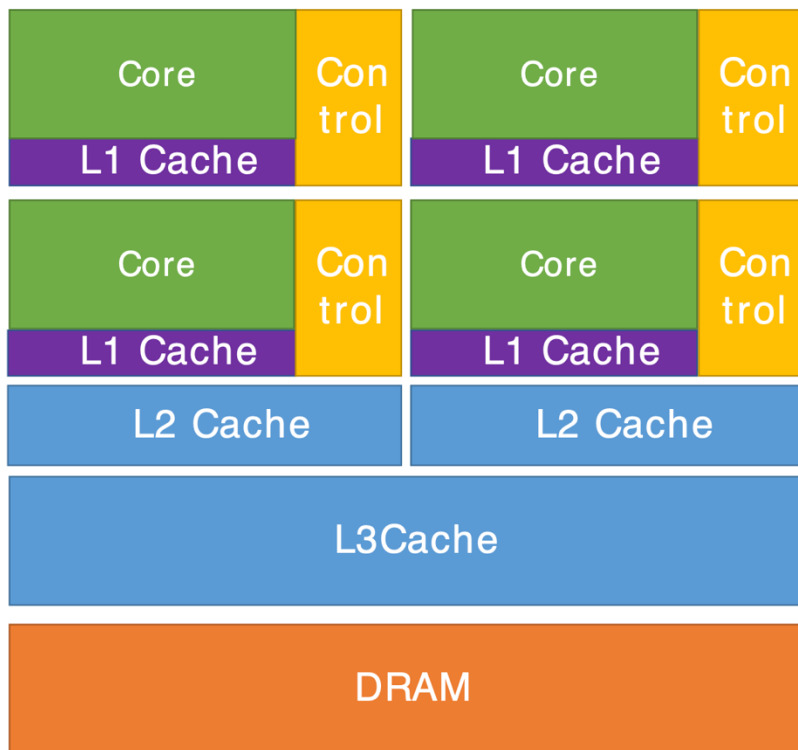
■ GPU硬件层次

- GPU
- Streaming Multiprocessor (SM)
- Warp
- Core

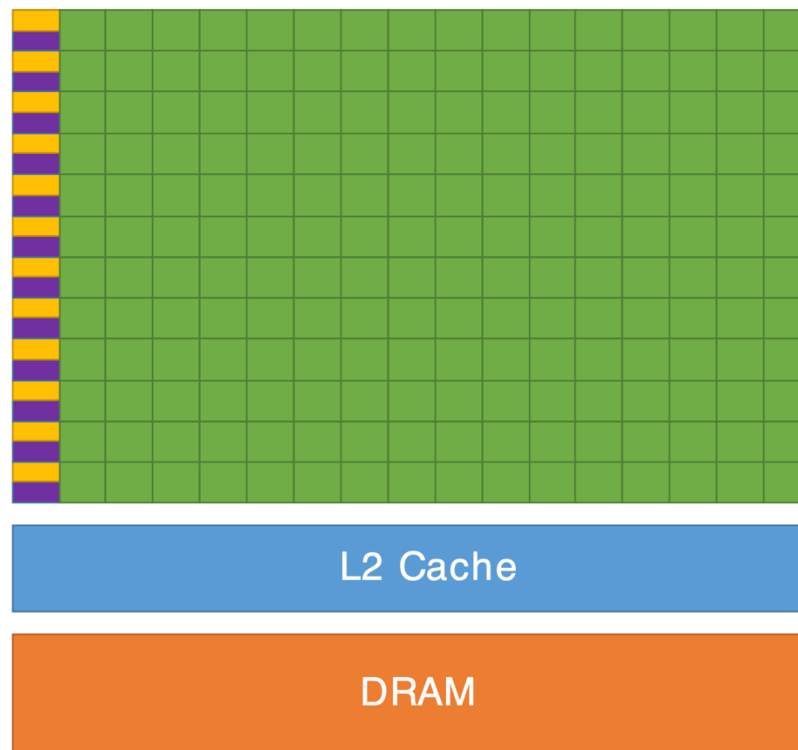


CPU 和 GPU 架构对比

- CPU：少量大核、单核计算能力强
- GPU：大量小核、单核计算能力弱



CPU



GPU

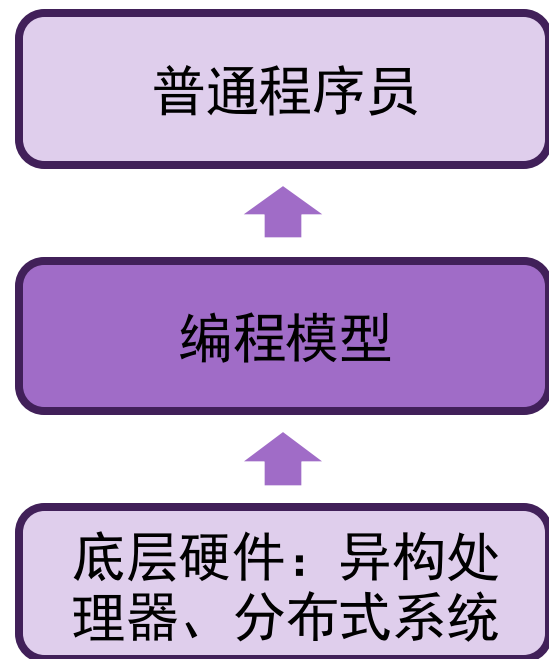
如何在 GPU 上设计高效编程模型

■ GPU 体系结构特点

- 大量计算核心
 - core
 - SM
- 多级存储单元
 - 寄存器
 - 共享存储
 - 全局内存

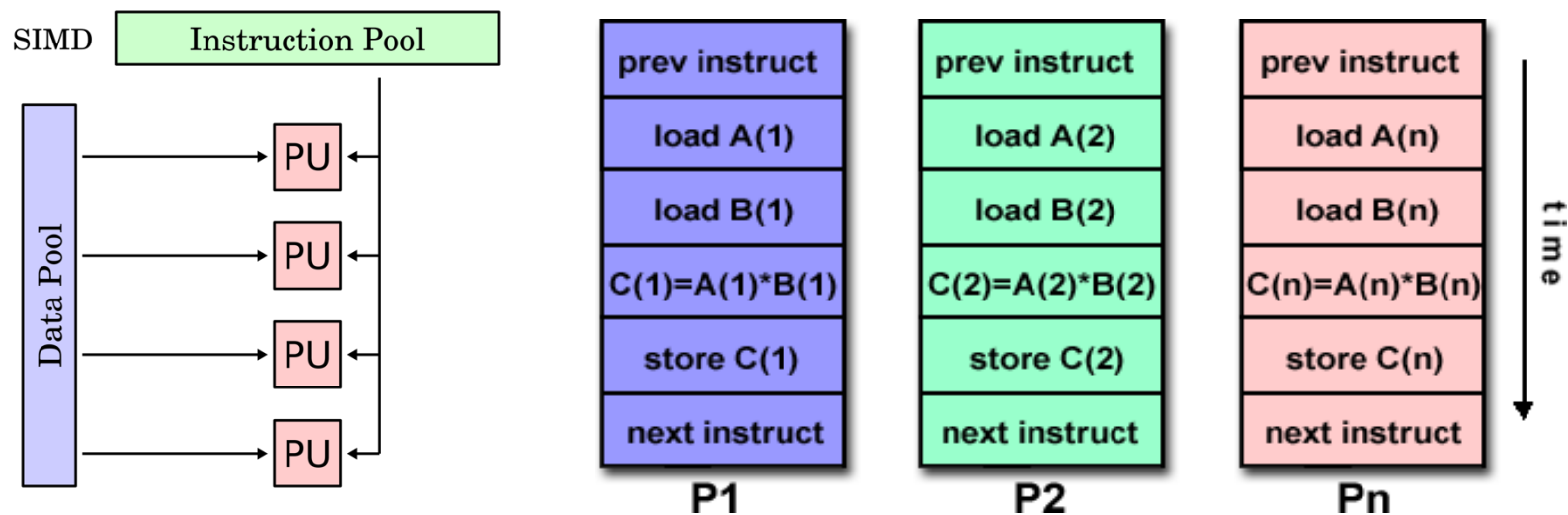
■ 面临挑战

- 如何设计一个易用编程模型？
- 如何充分发挥底层大量计算核心计算能力？
- 如何高效使用不同存储层次？



回顾：Flynn分类法：SIMD

- Single Instruction, Multiple Data (SIMD):
 - 单指令**: 所有处理单元同一时间都执行同一条指令
 - 多数据**: 不同处理单元处理不同数据
 - 例子: GPU、向量处理器（X86 AVX instruction）



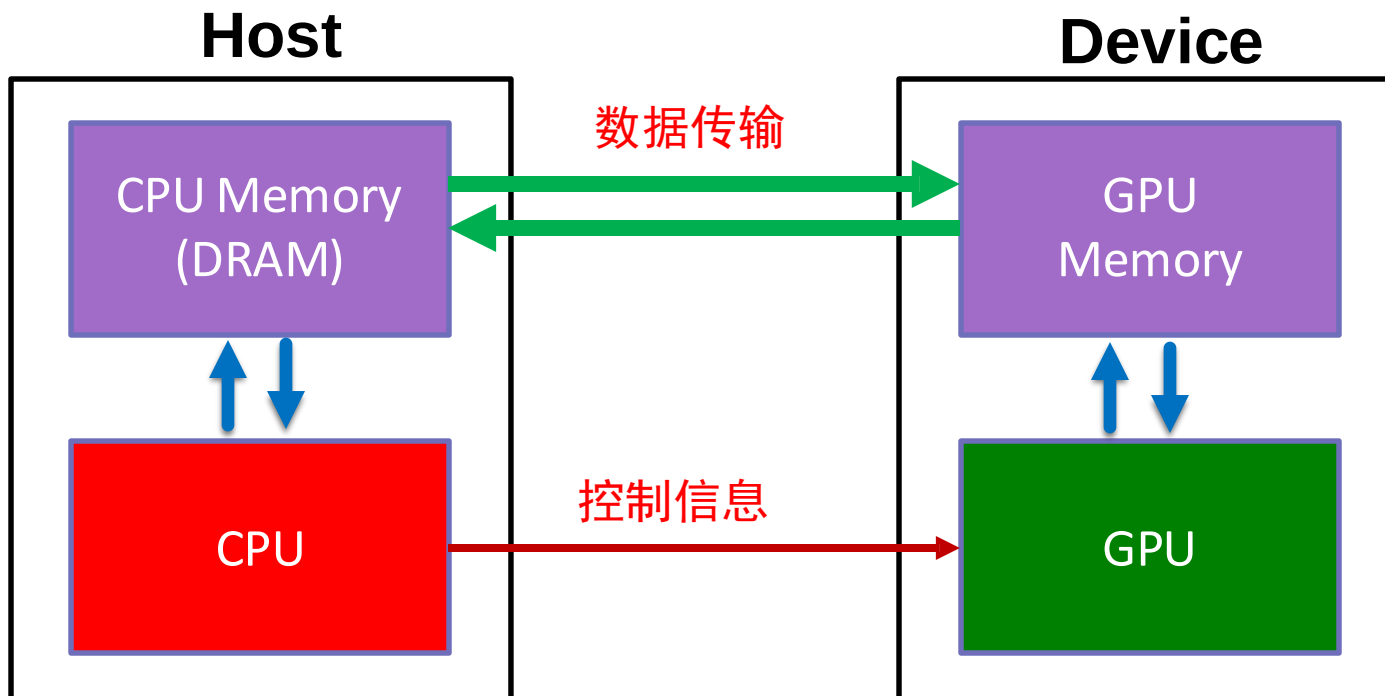
什么是 CUDA

- CUDA: Compute Unified Device Architecture
 - 2007年 Nvidia 公司引入通用计算编程模型
 - 在 NVIDIA GPU上编程使用的编译器和工具库
 - 可以发挥 GPU 的异构计算性能
- CUDA特点:
 - CUDA API 简单扩展了C/C++ 编程语言
 - 关键字、内嵌变量
 - 特殊符号
 - 库函数
 - SIMT 编程模型
 - SIMD + Multi-threading
 - 给硬件提供了更高层次的抽象



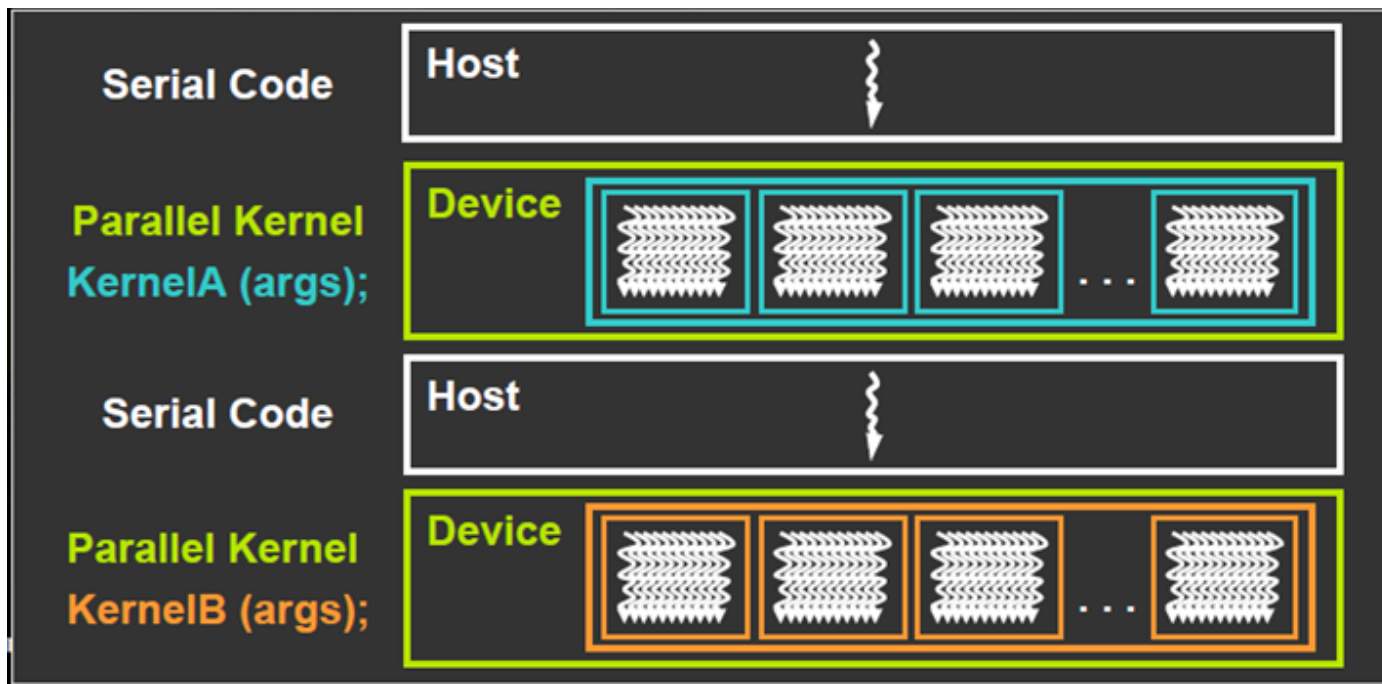
CUDA 编程基本概念

- 基本概念:
 - **Host:** 主机
 - **Device:** 设备 (GPU卡)



CUDA 编程模型

- **CUDA 程序 = Host 程序 + Device 程序**（也叫核函数 Kernel Function）
 - **Host 程序**在主机上执行 (如：CPU 线程)
 - **Device 程序**在 GPU 设备上并行执行 (如：GPU 线程)



CUDA 编程模型

■ Device 程序

- 核函数 (Kernel)
- 在 GPU 上执行
- `__global__` 开头

■ Host 程序

- 在 CPU 上执行
- 串行或并程序序
- OpenMP 或 MPI

■ Kernel Launch

- 一次 Kernel Launch, 创建大量 CUDA 线程
- 思考 openMP 线程

```
#include <cuda_runtime.h>
__global__ void my_kernel (...)
{
    ...
}

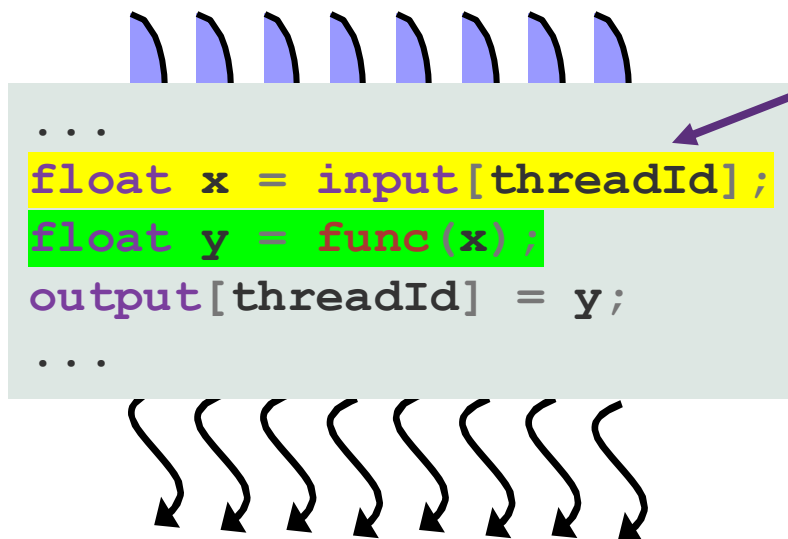
int main ()
{
    ...
    cudaMalloc (...);
    cudaMemcpy (...);
    ...
    my_kernel<<<nblock,blocksize>>>> (...);
    ...
    cudaMemcpy (...);
    ...
}
```

Kernel = 大量并发线程

- 在 GPU 上执行一个核函数
- 大量并发线程执行**同一个核函数**
 - SPMD 编程风格
 - 每个线程都执行**同一份代码**
 - 不同线程处理**不同数据**

ThreadId

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

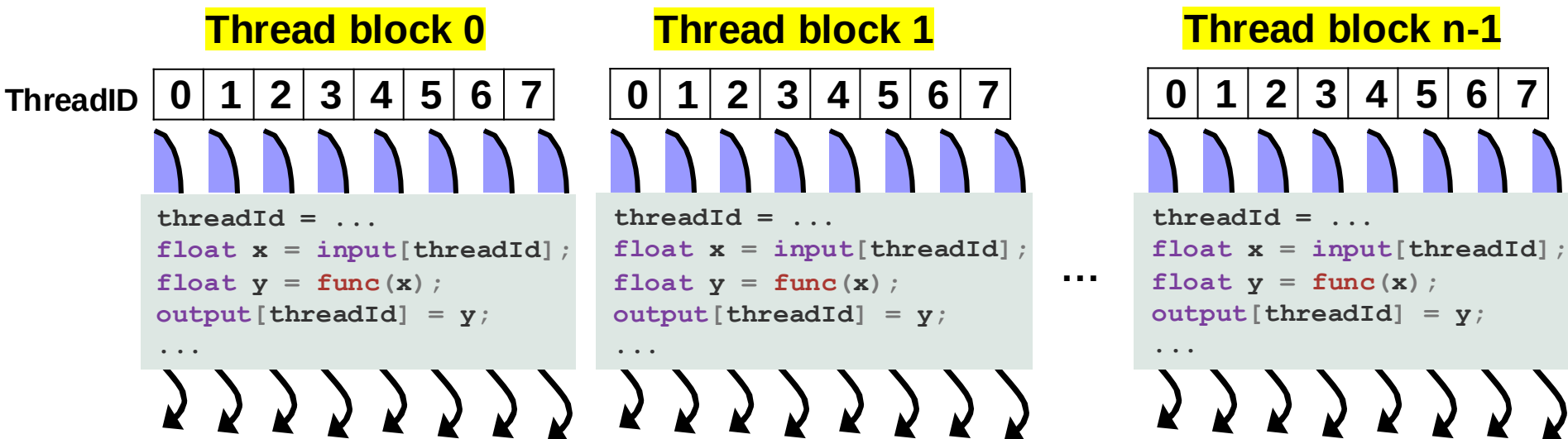


- 读取数据更灵活

每个线程都执行同一份代码、处理不同数据

线程如何组织 → 线程块

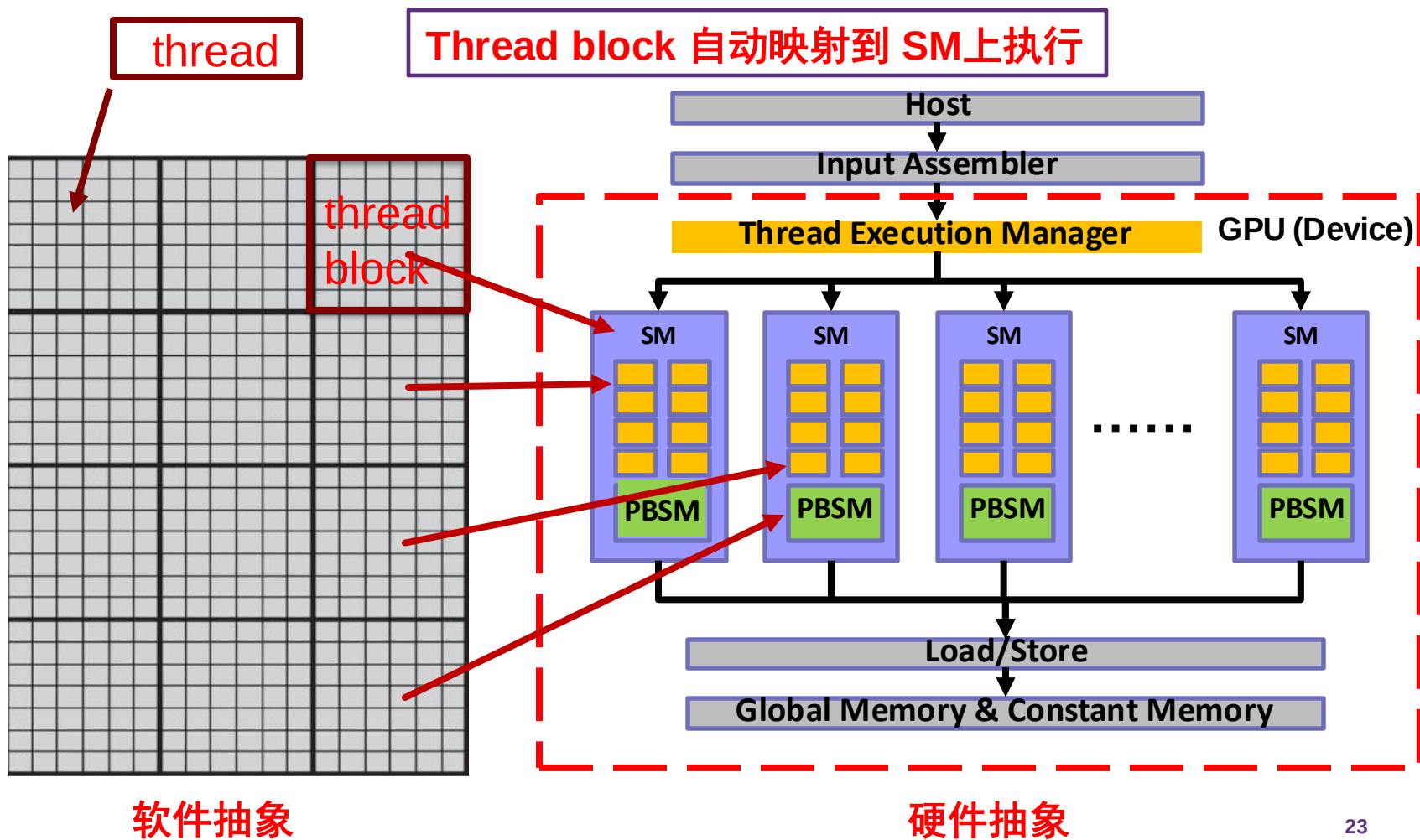
- 线程被分组到线程块中 (thread block)
 - Kernel 由大量线程块构成
 - 多个线程块构成一个 grid (Grid → Thread blocks → Threads)
 - kernel = grid of thread blocks
 - 每个线程块包括大量线程



思考：为什么要把线程分成线程块（OpenMP没有线程块）？

软件抽象与硬件的对应关系

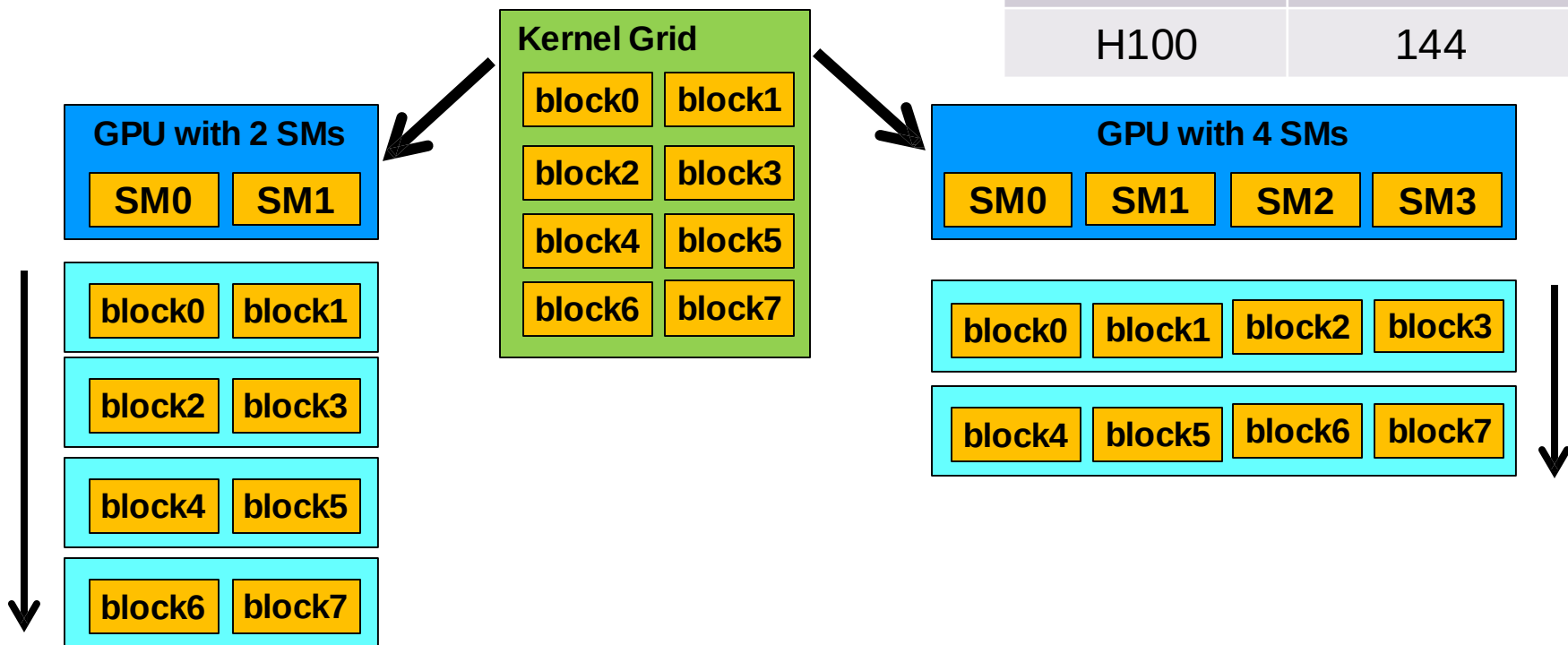
- 软件: grid $\rightarrow$ blocks $\rightarrow$ threads
- 硬件: GPU (设备) $\rightarrow$ SM (多核处理器) $\rightarrow$ Core (核心)



SM 与 Thread block: 透明扩展

- 程序员角度:
 - 不能对 thread block 的**执行顺序**有任何假设
 - 底层硬件会对 thread block 以**任意顺序**调度执行
- Thread block 特点:
 - **之间无依赖** → 可扩展性
 - 核函数可以扩展到任意 SM 上

GPU 名称	SM 数量
Tesla P100	56
Tesla V100	80
A100	108
H100	144



Thread block 和 Thread 的数量限制

- Compute capability: 描述硬件架构的计算能力和特性

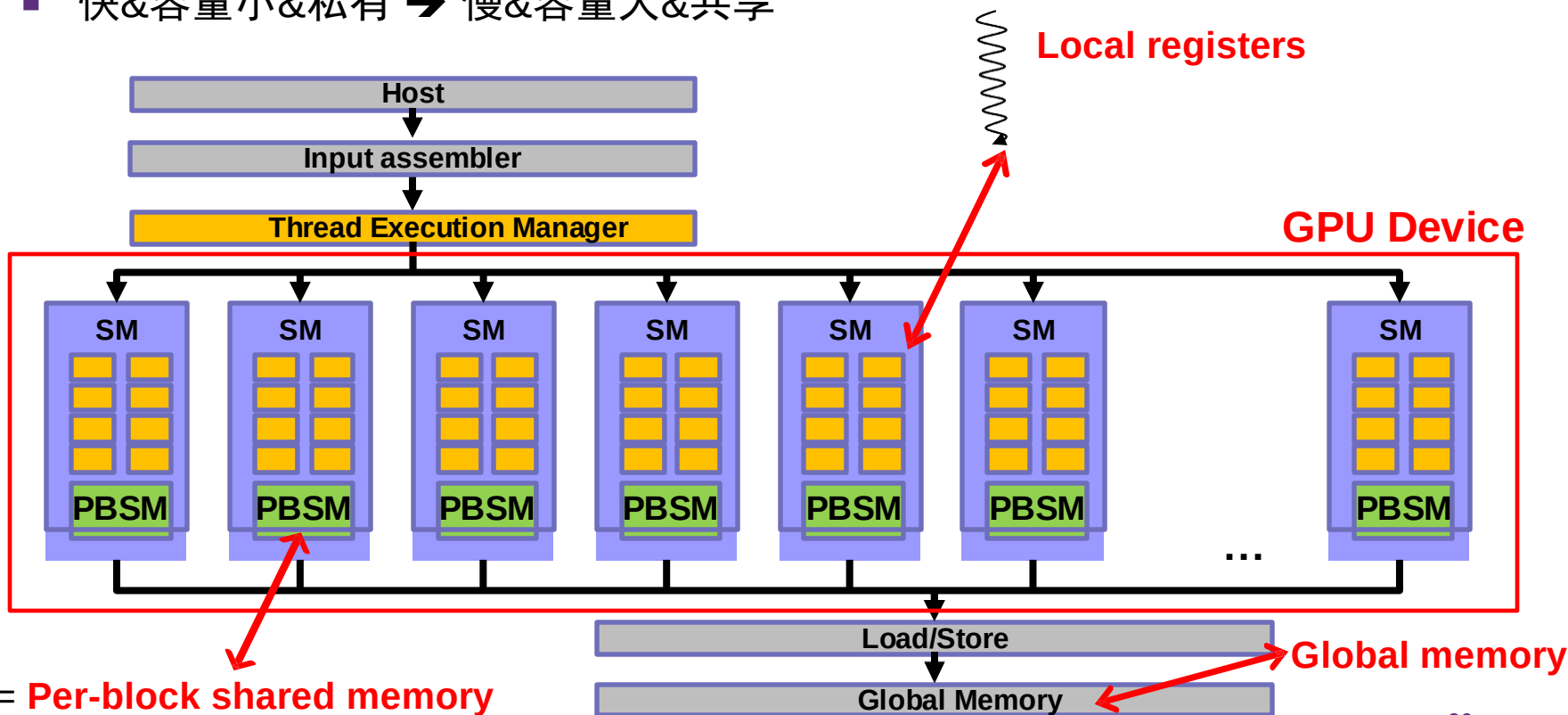
1.0	1.1	1.2	1.3	2.x	3.0	3.2	3.5, 3.7, 5.0, 5.2	5.3	6.x	7.x	8.0
-----	-----	-----	-----	-----	-----	-----	--------------------	-----	-----	-----	-----

- 每个 thread block 中**最多的线程数**
 - 在 Compute capability 2.x 之前: 512
 - 在 Compute capability 2.x 之后: 1024
- Thread block 数的**限制**
 - 在 Compute capability 3.0 之前: 65535
 - 在 Compute capability 3.0 之后: $2^{31} - 1$
- **线程总数 = threads per block * number of blocks**

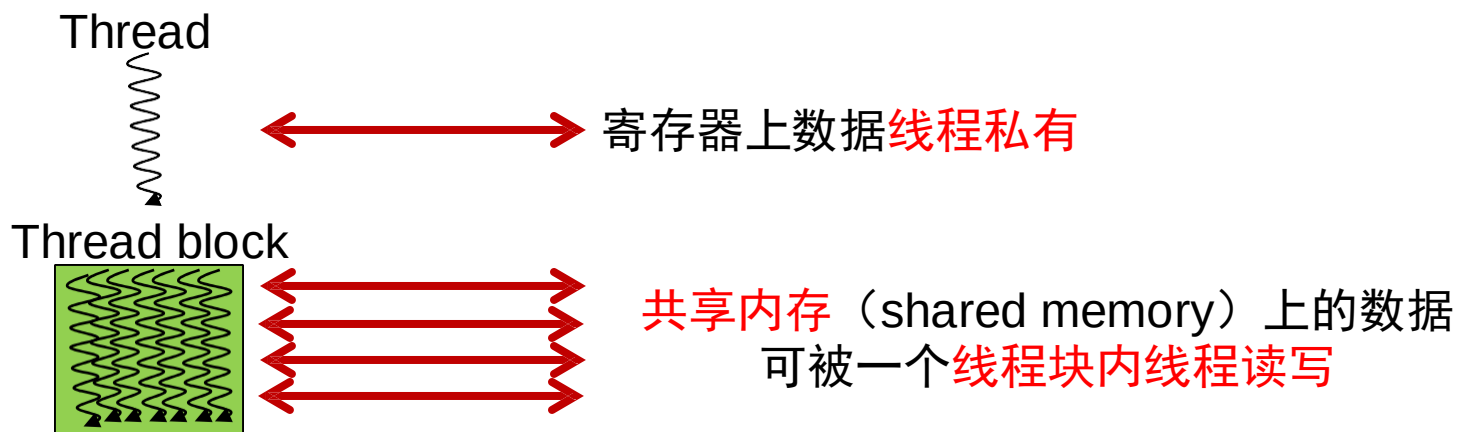
GPU 存储组织

■ 分层的存储组织:

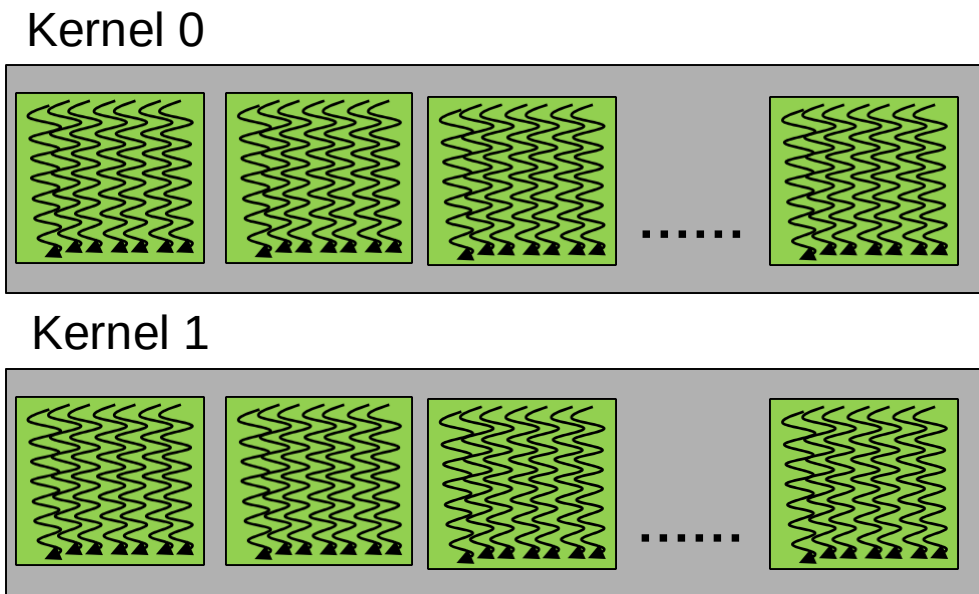
1. local register (寄存器) →
 2. shared memory (共享内存) →
 3. global memory (全局内存)
- 快&容量小&私有 → 慢&容量大&共享



数据是如何组织的？



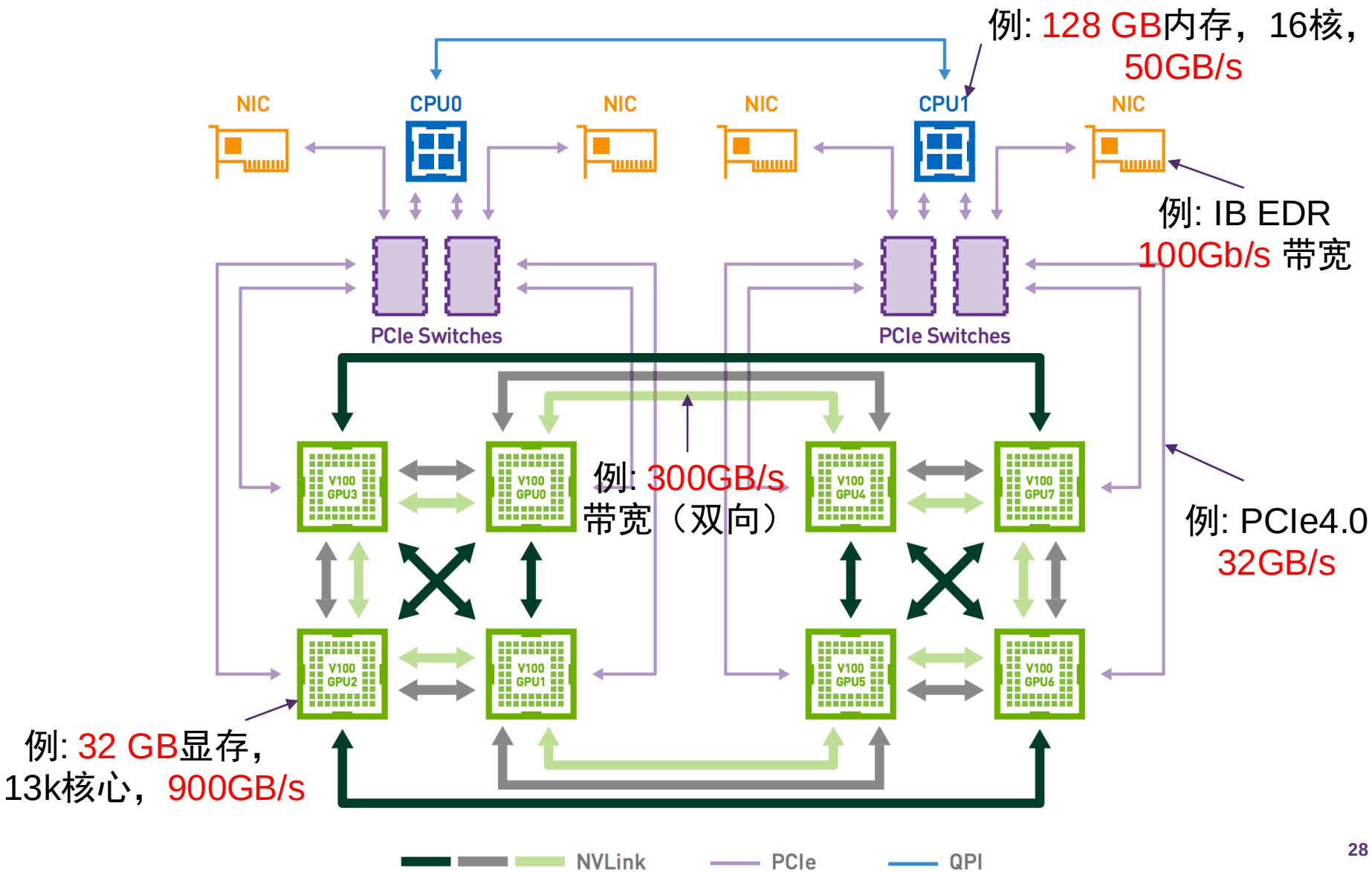
调用的
核函数



全局内存 (global memory) 上的数据
可被 **所有线程** 读写

A large red double-headed arrow points from the thread blocks to the global memory, indicating that all threads across all kernels can access this memory.

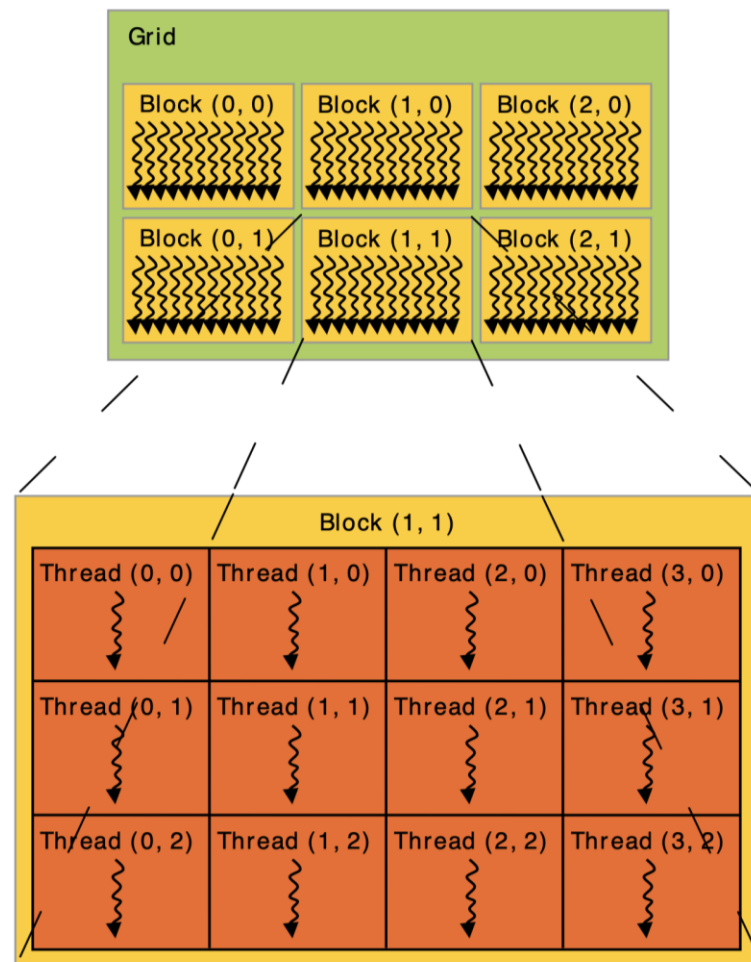
GPU 和 CPU 硬件组织



CUDA 规范与 API

CUDA 编程术语

- Host : CPU
- Device : GPU
- Kernel : GPU上执行的函数
- 线程 (Thread) :
 - 基本执行单元
- 线程块 (Thread Block) :
 - 包括一组线程
- Grid :
 - 包括一组线程块



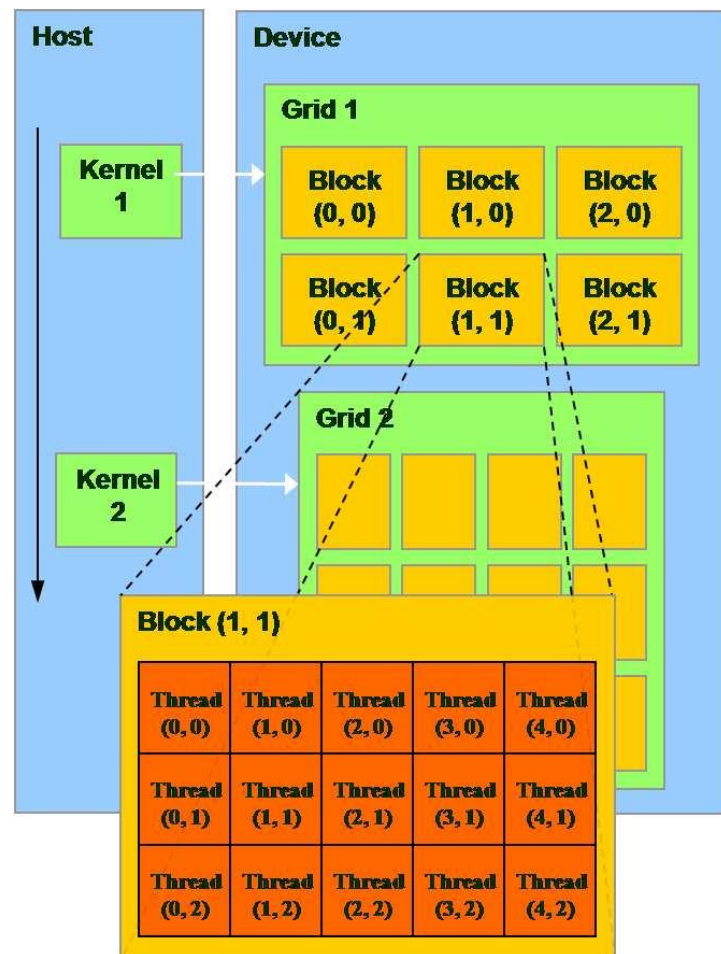
GPU 线程组织

CUDA 规范

- 启动核函数 (Kernel launch) : 创建大量线程
- `kernelFunc <<< nB, nT, nS, Sid >>> (...);` // nS 和 Sid 是可选
 - nB: 一个grid中的block数 (grid size)
 - nT: 一个block中的线程数 (block size)
 - nS: shared memory 大小 (单位为byte)
 - Sid: stream ID, 默认为0
- 预设的 device 变量
 - `threadIdx;`
 - `blockIdx;`
 - `blockDim;`

Thread ID 与 Block ID

- Threads 和 blocks 都有唯一 ID
 - 可以通过**预设变量**查询
 - `threadIdx`、`blockIdx`
 - `dim3` 类型 `struct dim3 {x; y; z;}`
- Thread IDs 是1, 2或3维
 - `threadIdx.x`; `threadIdx.y`;
`threadIdx.z`
- Block IDs 是1, 2或3维
 - `blockIdx.x`; `blockIdx.y`;
`blockIdx.z`
- 可以方便处理多维数据
 - 向量、矩阵、多面体
- 例子：右图
 - `dim3 grid (3, 2); // 6个block`
 - `dim3 blk (5, 3); // 15个线程`
 - `my_kernel<<<grid, blk>>>();`



创建了90个线程，每个线程都有自己唯一标识：<blockIdx.x, blockIdx.y; threadIdx.x, threadIdx.y>

CUDA 代码示例：向量相加

- 线程索引：一维（向量访问）
 - 通过 `threadIdx.x` 索引线程

```
// Kernel 定义
__global__ void VecAdd(float *A, float *B, float *C)
{
    int i = threadIdx.x;
    C[i] = A[i] + B[i]; //SPMD 编程风格
}

int main()
{
    ...
    //kernel 启动; 1 个 block、每个 block N 个线程
    VecAdd <<<1, N>>> (A, B, C);
    ...
}
```

CUDA 代码示例：矩阵相加

- 线程索引：二维（矩阵访问）
 - 通过 `threadIdx.x` 和 `threadIdx.y` 索引线程

```
__global__ void MatAdd(float **A, float **B, float **C)
{
    int i = threadIdx.x;
    int j = threadIdx.y;
    C[i][j] = A[i][j] + B[i][j];
}

int main()
{
    ...
    // Kernel 启动; 1 个 block、每个 block 包括 N*N 个线程
    dim3 threadsPerBlock(N, N); // 2 维 block
    MatAdd<<<1, threadsPerBlock>>>(A, B, C);
    ...
}
```

CUDA 代码示例：矩阵相加

- 线程索引
 - 块里面的线程数有限制（1024个）
 - 设置更多的线程块

```
__global__ void MatAdd(float **A, float **B, float **C)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    C[i][j] = A[i][j] + B[i][j];
}

int main()
{
    dim3 threadsPerBlock(16, 16); //划分成 16*16 块
    dim3 numBlocks(N/16, N/16);
    // 每个线程块 256 个线程, N*N/256 个线程块
    MatAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
}
```


线程同步

- 线程块内所有线程进行同步
 - `__syncthreads()` ;
 - 同步线程块中所有线程
- 注意:
 - 在同一线程块内的线程可以同步
 - 而线程块之间的线程不能同步

```
buffer[threadID] = memory[threadID];  
__syncthreads();  
int left = buffer[threadID - 1];
```

`MPI_Barrier(comm)`

函数和变量的类型

- 由于涉及 CPU 和 GPU 不同硬件
 - 函数会在不同硬件上执行
- 由于 GPU 复杂的存储层次
 - 变量会存储在不同存储单元
- 需要显示方法声明不同的函数和变量
 - 函数修饰符
 - 变量类型修饰符
- 例子:

- `__global__ void KernelFunc(...);` // Kernel 函数，在设备运行
- `__device__ void GlobalVar;` // 设备内存上变量
- `__shared__ void SharedVar;` // 共享内存上变量

函数修饰符

函数修饰符	限制
<code>__device__</code> function	在设备上执行 仅仅能在设备端调用
<code>__global__</code> function	在设备上执行 在 host 端调用 (只能是 void 返回类型)
<code>__host__</code> function	在 host 端执行 在 host 端调用
没有修饰符的函数	只为 host 端编译
<code>__host__ __device__</code> function	同时为 Host 端和 Device 端编译

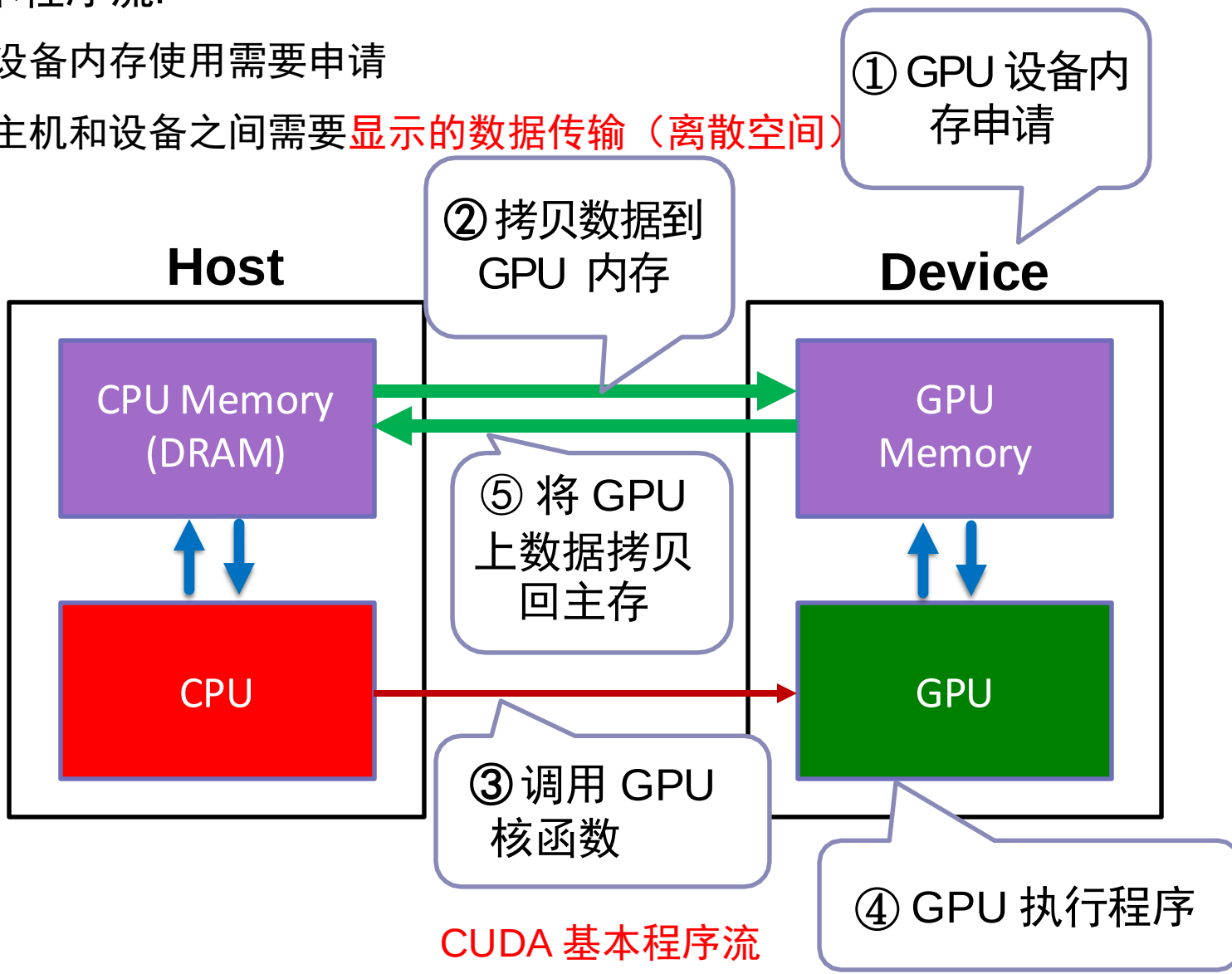
变量类型修饰符

变量修饰符	限制
<code>__device__</code> var	➤ 变量存在于device 的 global memory ➤ 所有线程都可以读写 ➤ 读写性能差
<code>__shared__</code> var	➤ 变量存在于共享内存 shared memory ➤ 与线程块的生命周期相同 ➤ 同一个线程块中的线程可以读写 ➤ 读写性能好
<code>__constant__</code> var	➤ 变量存在于 constant memory 中 ➤ 与应用的生命周期相同 ➤ 所有线程都可以读写，也可以用 runtime library 从 host 端访问

CUDA 基本程序流

■ 基本程序流:

- 设备内存使用需要申请
- 主机和设备之间需要显示的数据传输（离散空间）



如何操作 GPU device memory

- 3个函数:

- `cudaMalloc()`, `cudaFree()`, `cudaMemcpy()`
- 与 C 的 `malloc()`, `free()`, `memcpy()` 相似

1. `cudaMalloc(void **devPtr, size_t size)`

- `devPtr`: 返回申请到的 device memory 的地址
- `size`: 需要申请的内存大小 (bytes)

2. `cudaFree(void *devPtr)`

3. `cudaMemcpy(void *dst, const void *src, size_t count, enum cudaMemcpyKind kind)`

- `count`: 要拷贝的内存大小 (bytes)

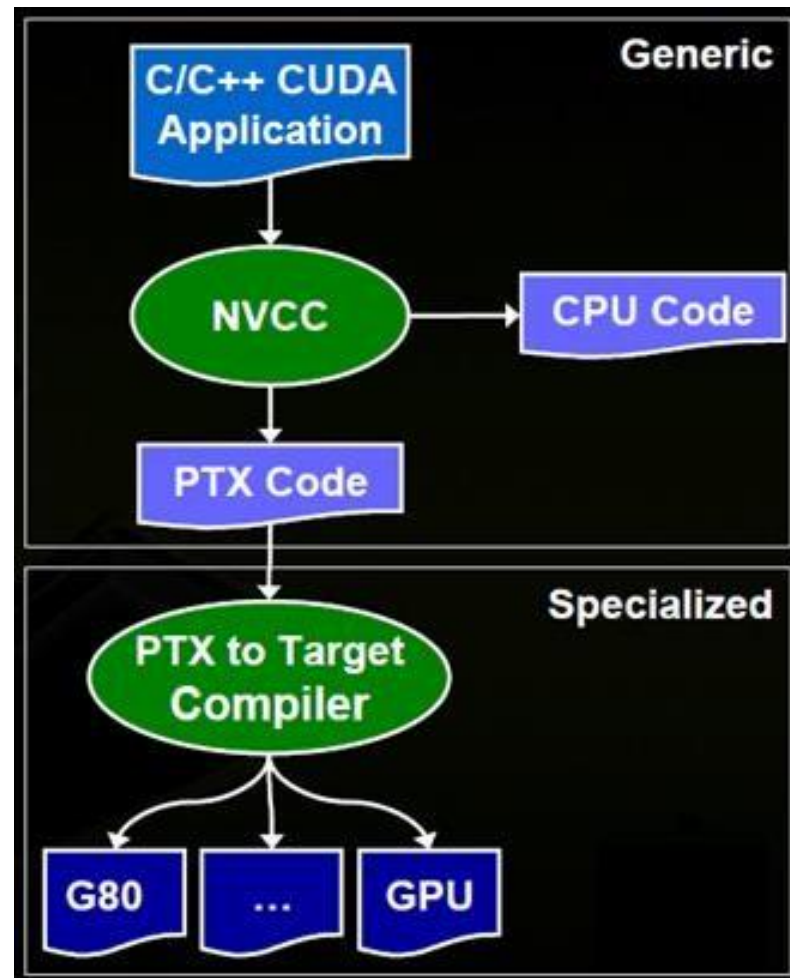
CPU 和 GPU 之间内存传输

- 设置内存拷贝的方向
- 其中 host to host 与 `memcpy()` 等价
- 设备间消息传递

<code>cudaMemcpyKind</code>	含义	目的	源
<code>cudaMemcpyHostToHost</code>	Host → Host	host	host
<code>cudaMemcpyHostToDevice</code>	Host → Device	device	host
<code>cudaMemcpyDeviceToHost</code>	Device → Host	host	device
<code>cudaMemcpyDeviceToDevice</code>	Device → Device	device	device

CUDA 程序编译过程

- 含有 CUDA 语言的源文件都需要用 NVCC 编译
 - NVCC 会将在 device 端和在 host 端的代码分离
- GPU 端两阶段编译过程：
 - 设备无关编译（Generic）
 - `nvcc --ptx -o a.ptx a.cu`
 - 设备相关编译（Specialized）
 - `nvcc -gencode arch=compute_70,code=sm_70 -o a.out a.cu`
 - PTX: Parallel Threads eXecutions
 - 设备相关的二进制对象
 - 虚拟指令集（方便编译优化）



CUDA 例子

例子1: Hello World

```
__global__ void mykernel(void)
{
    printf("Hello World!\n");
}
int main(void)
{
    mykernel<<<1,1>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

■ 语法解释

- `__global__` 指运行在 device 上的函数，可以在 host 端被调用
- Kernel 启动: `mykernel<<<1,1>>>()` ;
 - 三个尖括号指使用 host 代码调用 device 函数，一次 kernel launch
 - 使用1个线程块，每个线程块1个线程

例子2：两数之和

```
__global__ void add(int *a, int *b, int *c)
{
    *c = *a + *b;
}
int main(void)
{
    int ha=1,hb=2,hc;
    add<<<1,1>>>(&ha, &hb, &hc);
    printf("c=%d\n",hc);
    return 1;
}
```



- 错误原因： `int ha=1,hb=2,hc` 在 host memory 中, 不能被 device 访问、数据是分离的
- 需要在 device memory 中申请内存

正确的实现

```
int main(void)
{
    int a=1, b=2, c; // CPU 上的变量
    int *d_a, *d_b, *d_c;
    // 在 GPU 设备上分配变量空间 a, b, c
    cudaMalloc((void **)&d_a, sizeof(int));
    cudaMalloc((void **)&d_b, sizeof(int));
    cudaMalloc((void **)&d_c, sizeof(int));
    // 从 CPU 拷贝数据到 GPU
    cudaMemcpy(d_a, &a, sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, &b, sizeof(int), cudaMemcpyHostToDevice);
    // 在 GPU 上启动 Kernel: 1 个线程
    add<<<1,1>>>(d_a, d_b, d_c);
    // 从 GPU 上拷贝结果回 CPU
    cudaMemcpy(&c, d_c, size, cudaMemcpyDeviceToHost);
    // 释放内存
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}
```

例子3：向量加法

- CPU 串行代码：

```
// 计算  $A + B = C$   
void VecAdd(int N, float *A, float *B, float *C)  
{  
    for(int i = 0; i < N; i++)  
        C[i] = A[i] + B[i];  
}  
int main()  
{  
    ...  
    VecAdd (N, Ah, Bh, Ch);  
    ...  
}
```

第一种实现：向量加法

- 使用1 个线程块、多个线程
- 以 `threadIdx.x` 作为数组索引
 - 每个线程处理一条加法（第 `threadIdx.x` 个加法）

```
// Kernel 定义
__global__ void VecAdd(float *A, float *B, float *C)
{
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}

int main()
{
    ...
    // 创建1个 thread block, 每个 block N 个线程
    VecAdd<<<1, N>>>>(Ah, Bh, Ch);
    ...
}
```

第二种实现：向量加法

- 使用多个线程块
 - N 个 blocks, 每个 block 有一个线程
- 两种方法孰优孰劣?
 - 线程块内线程数有限制（1024个）
 - 相同 block 内线程之间可以交换数据和同步

```
__global__ void add(int *a, int *b, int *c)
{
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];
}
int main()
{
    int a[N], b[N], c[N];
    int *d_a, *d_b, *d_c;

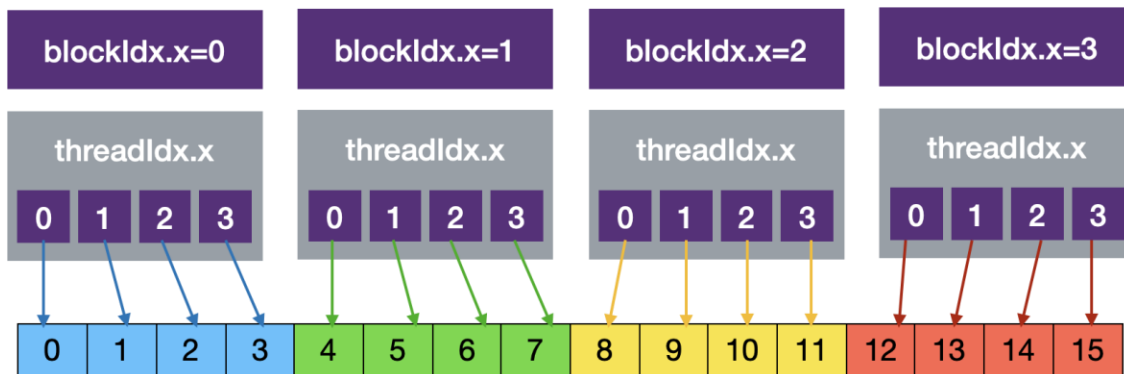
    // 创建 N 个 thread block, 每个 block 1个线程
    add<<< N, 1 >>>(d_a, d_b, d_c);
}
```

第三种实现：向量加法

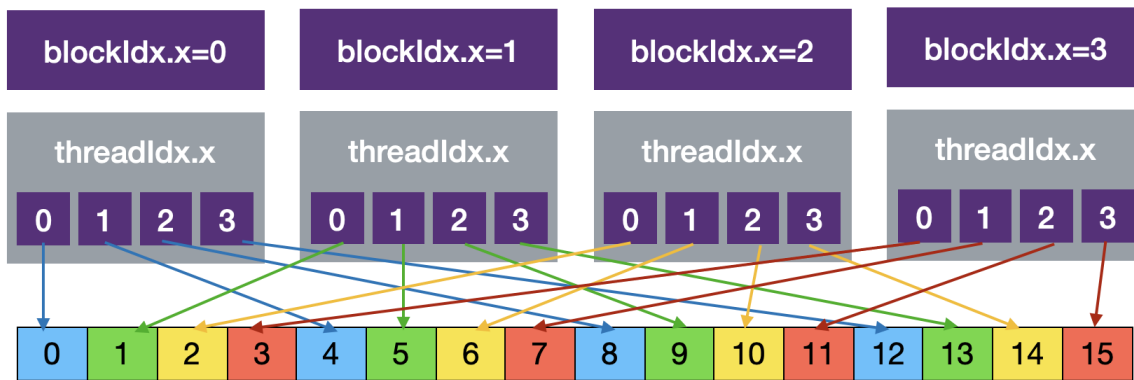
- 多个线程，多个 block
- 若 $N=16$, 可以设 $\text{grid size} = 4$, $\text{block size} = 4$
- 如何把线程映射到要处理的 16 个元素？
 - 方法 1: $\text{index} = \text{threadIdx.x} + \text{blockIdx.x} * 4$ (一个thread block连续4个线程处理相邻元素)
 - 方法 2: $\text{index} = \text{blockIdx.x} + \text{threadIdx.x} * 4$ (每个thread block相同位置的线程处理相邻元素)

第三种实现 - 示意图

- 方法 1: $\text{index} = \text{threadIdx.x} + \text{blockIdx.x} * 4$

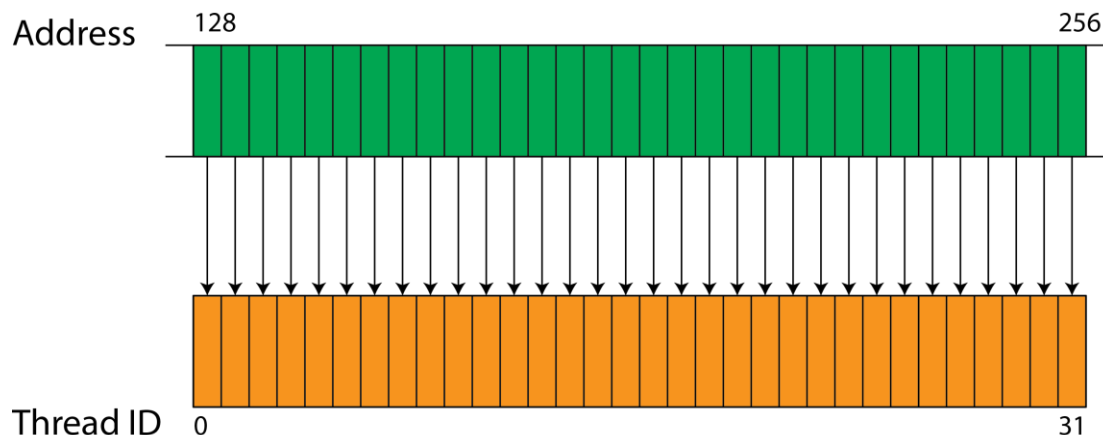


- 方法 2: $\text{index} = \text{blockIdx.x} + \text{threadIdx.x} * 4$



第三种实现：向量加法

- 多个线程，多个 block
- 若 $N=16$, 可以设 $\text{grid size} = 4$, $\text{block size} = 4$
- 如何索引16 个元素？
 - 方法 1: $\text{index} = \text{blockIdx.x} * 4 + \text{threadIdx.x}$
 - 方法 2: $\text{index} = \text{threadIdx.x} * 4 + \text{blockIdx.x}$
- 哪种实现更好？
 - GPU 的合并访存（Coalesced memory access）
 - 相临 32 个线程同时访问 128byte 数据（ $[128*n, 128*n+127]$ ）仅需一个 cycle



更普适的实现

- 使用预定义变量 `blockDim.x` 来访问每个线程块中的线程个数
- BS 是 block size（线程块大小）

```
__global__ void add(int *a, int *b, int *c)
{
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    c[index] = a[index] + b[index];
}

int main()
{
    int a[N], b[N], c[N];
    int *d_a, *d_b, *d_c;

    // 创建 N/BS 个 thread block, 每个block 有 BS个线程
    add<<<N/BS, BS>>>>(d_a, d_b, d_c);
    ...    当N不整除BS的时候?
}
```

更普适的实现

- Kernel 内部可以有分支，不满足分支条件的线程直接退出

```
__global__ void add(int *a, int *b, int *c, int n)
{
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    // 增加额外的判断，多余的线程不再执行
    if (index < n)
        c[index] = a[index] + b[index];
}

int main()
{
    int a[N], b[N], c[N];  int *d_a, *d_b, *d_c;
    // 创建更多的线程
    add<<<(N+BS-1)/BS, BS>>>(d_a, d_b, d_c, N);
    ...
}
```

例子4: shared memory 使用

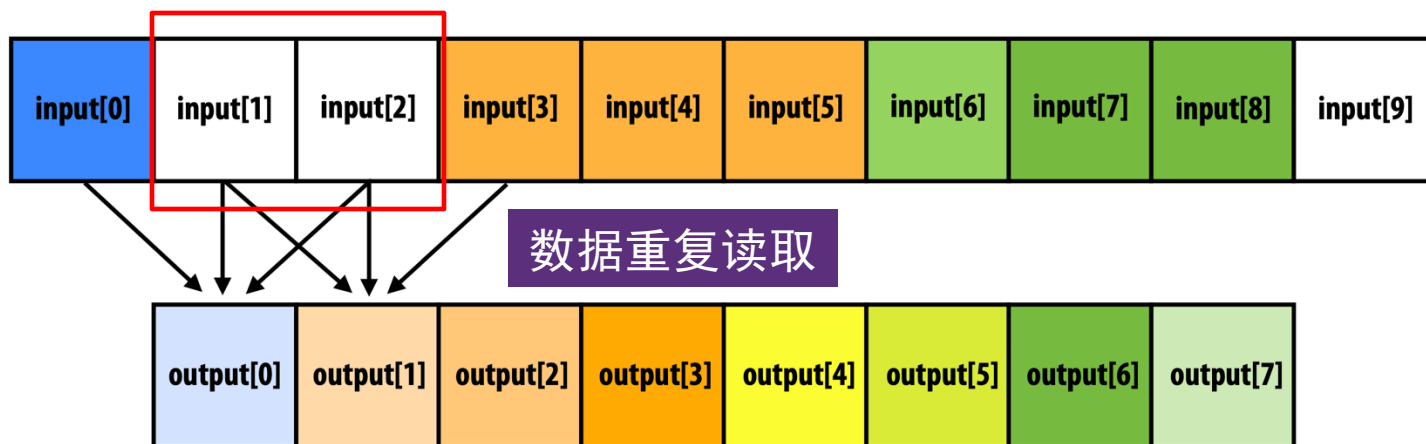
- shared memory 适合用于在核函数中被多次重复使用的数据

```
__global__ void computation_shared_mem(float *arr)
{
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    // 定义 shared memory
    __shared__ float shared_arr[64];
    int index_in_block = threadIdx.x;
    shared_arr[index_in_block] = arr[index];
    // 之后多次使用 shared_arr 进行计算
}

int main()
{
    ...
    computation_shared_mem <<< (N+BS-1)/BS, BS >>>(arr);
    ...
}
```

例子4：简单的 stencil

- 数组中每个元素，都需要加上其左邻居和右邻居
- 基础实现
 - 每个线程涉及三次对 global memory 的读取



```
__global__ void baseline(float *arr)
{
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    out[index-1] = input[index-1] + input[index] + input[index+1];
}
```

利用 shared memory 减少访存

■ 优化版本：

- 为每个线程块申请 64 个 float 大小的 shared memory
- 首先所有线程一起读取对应数据，并存入 shared memory
- 利用 `__syncthreads()`；同步线程块内线程，确保读取都已经完成
- 最后使用 shared memory 中数据进行计算（共享内存数据被重复使用）

```
__global__ void opt_with_shared_mem(float *arr)
{
    // Block size = 64 threads.
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    int index_in_block = threadIdx.x;
    __shared__ float shared_arr[64];
    shared_arr[index_in_block] = input[index];
    __syncthreads();
    output[index-1] = shared_arr[index_in_block-1] +
    shared_arr[index_in_block] + shared_arr[index_in_block+1];
}
```

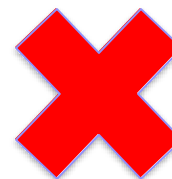
共享内存 `shared_arr[]` 数组的数据被反复使用，提高性能

CUDA 函数计时和调试

异步执行

- CUDA 函数计时
 - 执行到下一条CPU指令不等于 GPU kernel 执行结束

```
auto time_begin = timestamp();  
kernel <<< >>> ();  
auto time_end = timestamp();  
auto time_cost = time_end - time_begin;
```



异步执行

■ CUDA 函数计时

- 执行到下一条CPU指令不等于 GPU kernel 执行结束
- 需要在 CPU 上显式同步，以等待 GPU 函数执行结束

```
auto time_begin = timestamp();  
kernel <<< >>> ();  
auto time_end = timestamp();  
auto time_cost = time_end - time_begin;
```



```
auto time_begin = timestamp();  
kernel <<< >>> ();  
cudaDeviceSynchronize();  
auto time_end = timestamp();  
auto time_cost = time_end - time_begin;
```

显式同步

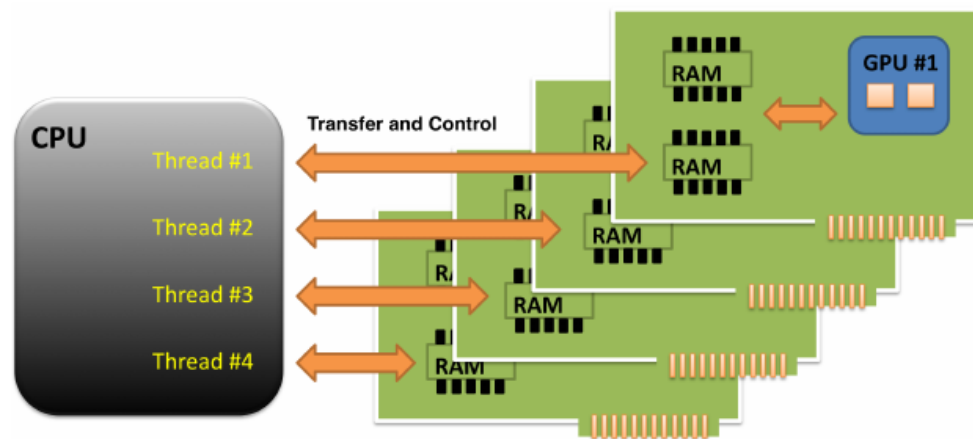
CUDA 错误码

- 所有 CUDA 相关调用都会返回 error code
 - `cudaError_t`
 - 可能是 API 本身的错误
 - 也可能是之前异步调用中的错误
- 从最后一个错误中得到错误码
 - `cudaError_t cudaGetLastError(void)`
- 通过得到字符串来描述错误
 - `char *cudaGetErrorString(cudaError_t)`

多 GPU 编程与动态并行

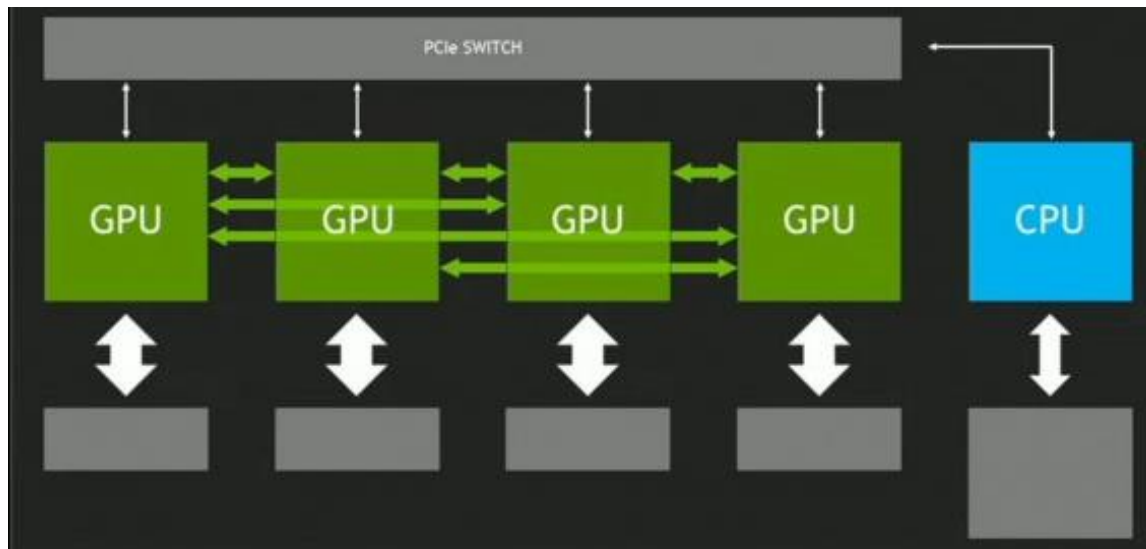
多 GPU 架构

- 节点内包括多个 GPU
- 在 CPU 端使用 `cudaSetDevice (int devID)`
 - 改变所使用的 GPU ID
- GPU 之间通信
 - 通过 `cudaMemcpy` 以及 `cudaMemcpyDeviceToDevice` 进行消息传输



OpenMP 和 CUDA

- 将 CUDA 函数放入并行区域
- 典型设置:
 - CPU 线程数与 GPU 数相同
 - 每个 CPU 线程控制一个 GPU 设备
- 也可以使用多于 GPU 数的 CPU 线程数
 - 多个 CPU 线程能向同一个 GPU 设备启动函数
 - 但可能会导致性能下降：资源竞争



OpenMP 和 CUDA 示例

```
cudaGetDeviceCount (&num_gpus) ;
omp_set_num_threads (num_gpus) ;
// 创建和 GPU 设备数相同的 CPU 线程数
#pragma omp parallel
{
    unsigned int cpu_thread_id =    omp_get_thread_num() ;
    unsigned int num_cpu_threads = omp_get_num_threads() ;
    CUDA_SAFE_CALL(cudaSetDevice(cpu_thread_id)) ;
    int gpu_id = -1;
    CUDA_SAFE_CALL(cudaGetDevice(&gpu_id)) ;

    printf("CPU thread %d (of %d) uses CUDA device %d\n",
        cpu_thread_id, num_cpu_threads, gpu_id);

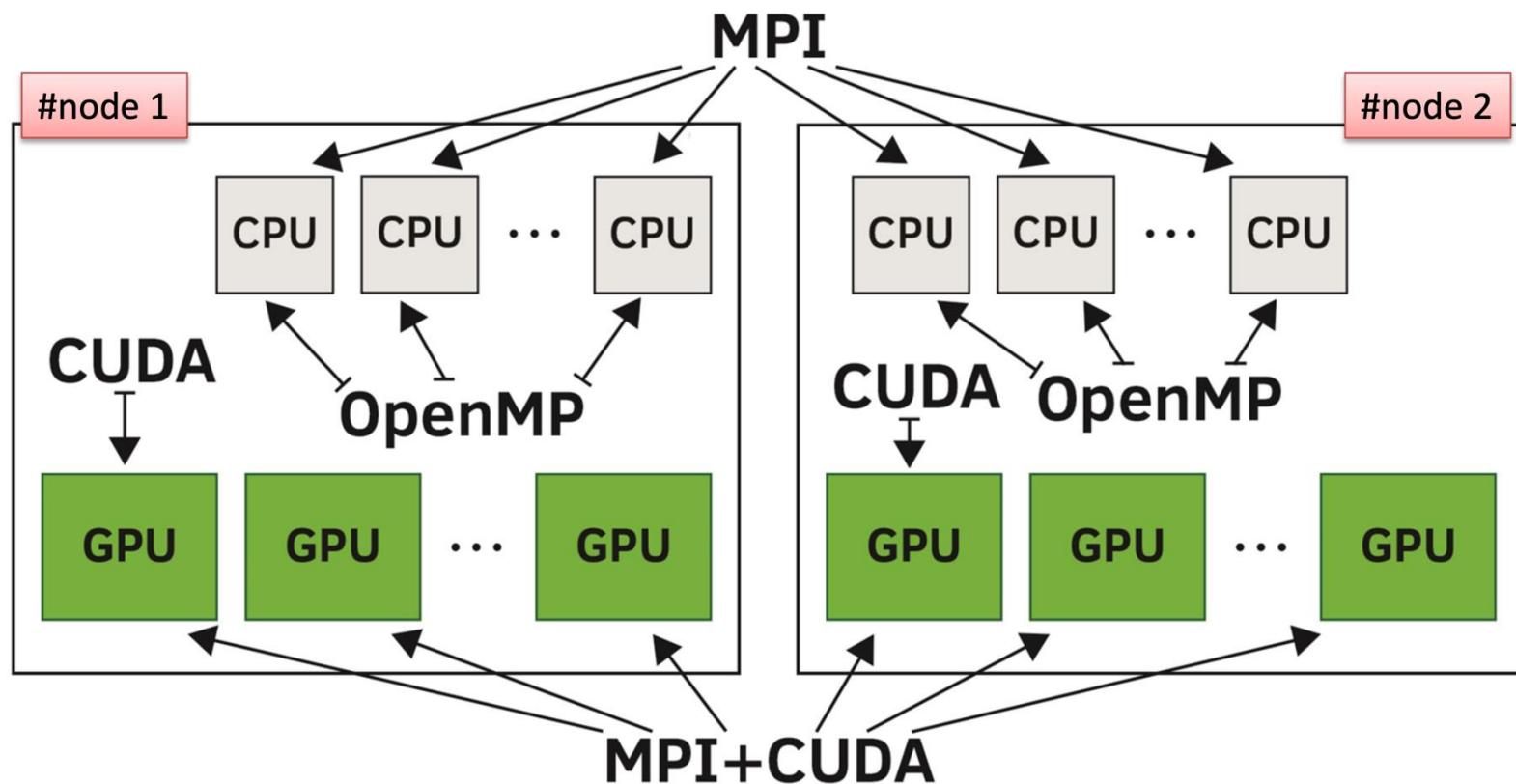
    // 在对应 GPU 设备启动 Kernel
    kernel <<< >>> () ;

    ...
}
```

主程序是一个多线程程序

MPI 和 CUDA

- 对于多机（集群环境）、每个服务器包括多个 GPU 设备
 - 采用 **MPI + OpenMP + CUDA** 混合编程



MPI + OpenMP + CUDA 混合编程

MPI 和 CUDA 示例

- 多进程调用核函数

```
int main(int argc, char* argv[])
{
    int rank, size;
    int A[32];  int i;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    printf("I am %d of %d\n", rank, size);
    for(i = 0; i < 32; i++)
        A[i] = rank+1;

    // 每个进程启动一个 Kernel
    launch(A);

    MPI_Barrier(MPI_COMM_WORLD);
    MPI_Finalize();
    return 0;
}
```

MPI 和 CUDA 示例

■ CUDA 核函数

```
extern "C" void launch(int *A)
{
    int *dA;
    cudaMalloc((void**)&dA, sizeof(int)*32);
    cudaMemcpy(dA, A, sizeof(int)*32, cudaMemcpyHostToDevice);

    // 在 GPU 设备上启动 32 个线程
    kernel<<<1, 32>>>(dA);

    cudaMemcpy(A, dA, sizeof(int)*32, cudaMemcpyDeviceToHost);
    cudaFree(dA);
}
```

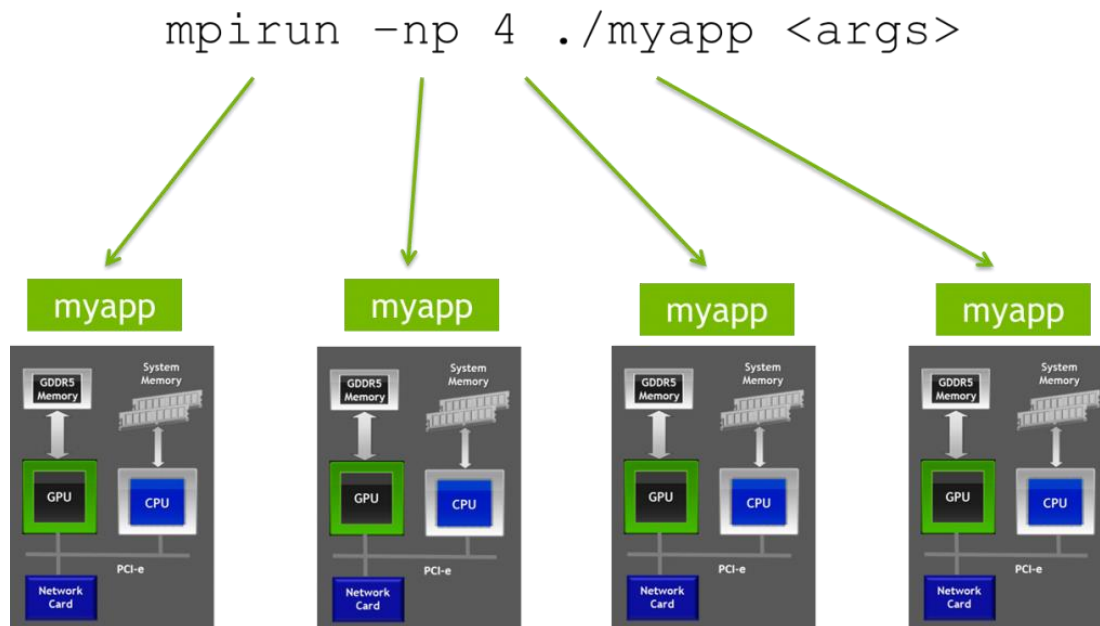
编译与运行

■ 程序编译

- `nvcc -c kernel.cu`
- `mpicc -o myapp mpi.c kernel.o -L /PATHTOCUDA/lib -lcudart -I /PATHTOCUDA/include`

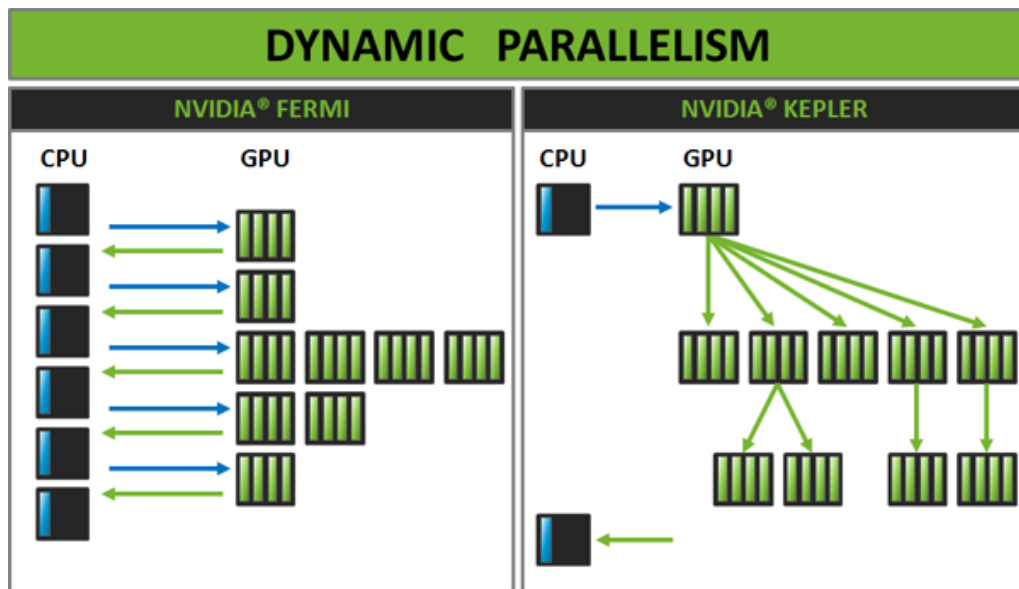
■ 程序运行

- `mpirun -np 4 ./myapp`



动态并行（Dynamic parallelism）

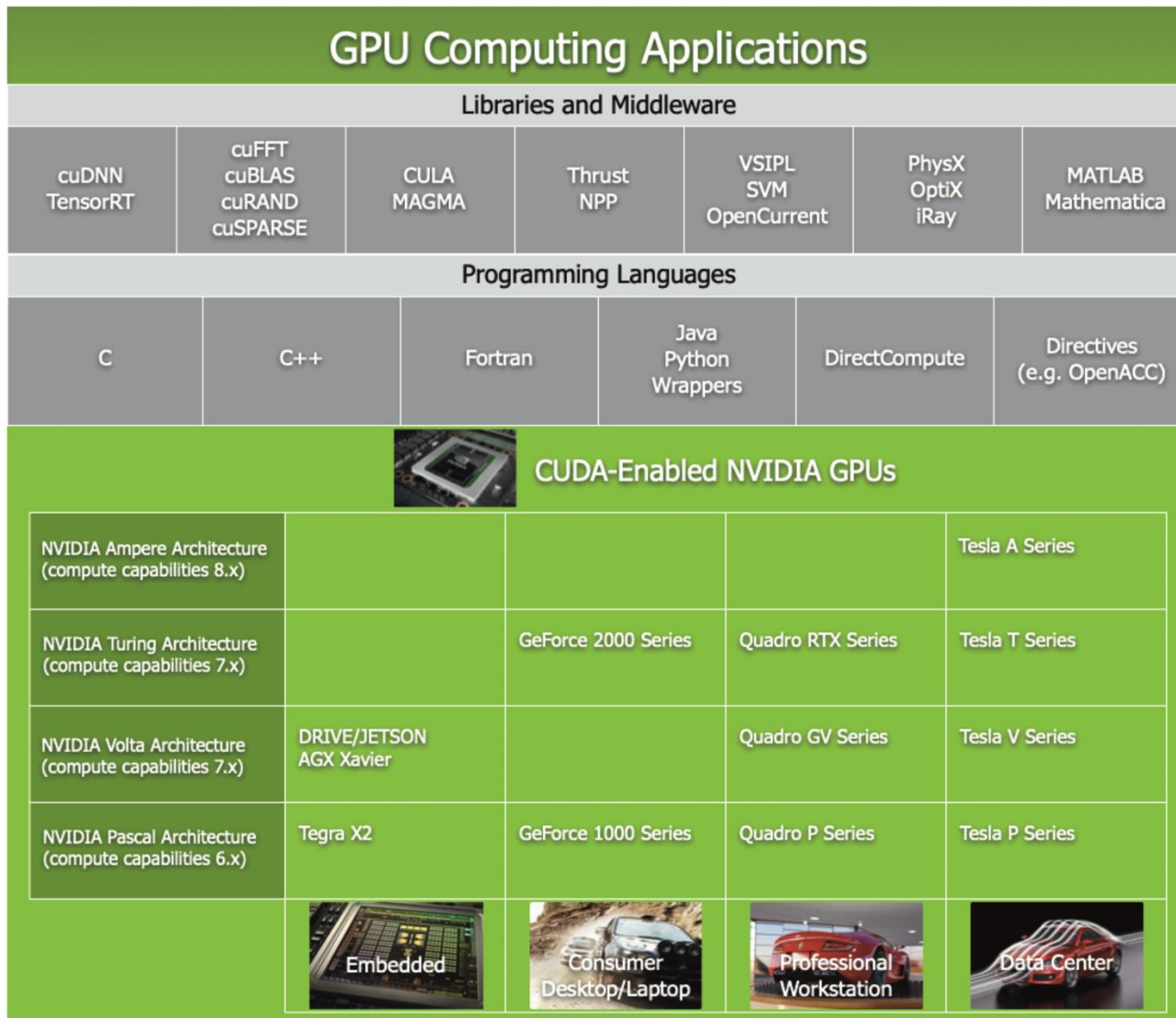
- GPU 支持在 GPU kernel 内再**动态创建**新的 kernel
 - 动态负载
 - 嵌套并行
 - 库函数调用
 - 递归调用
- 从 CUDA 5.0 之后，在 Compute Capability 3.5 之后的设备上开始支持，**性能一般**



GPU 编程生态

GPU 编程生态环境

■ GPU 编程生态：



GPU 计算库

- cuBLAS:
 - GPU 线性代数计算库
- cuSparse:
 - GPU 稀疏操作计算库
- cuDNN:
 - 为深度学习算子定制的GPU计算库
- Thrust:
 - STL for CUDA
- NCCL:
 - GPU通信库，支持机内/多机GPU通信
- TensorRT:
 - 高性能 GPU 推理（inference）框架



AMD GPU 编程

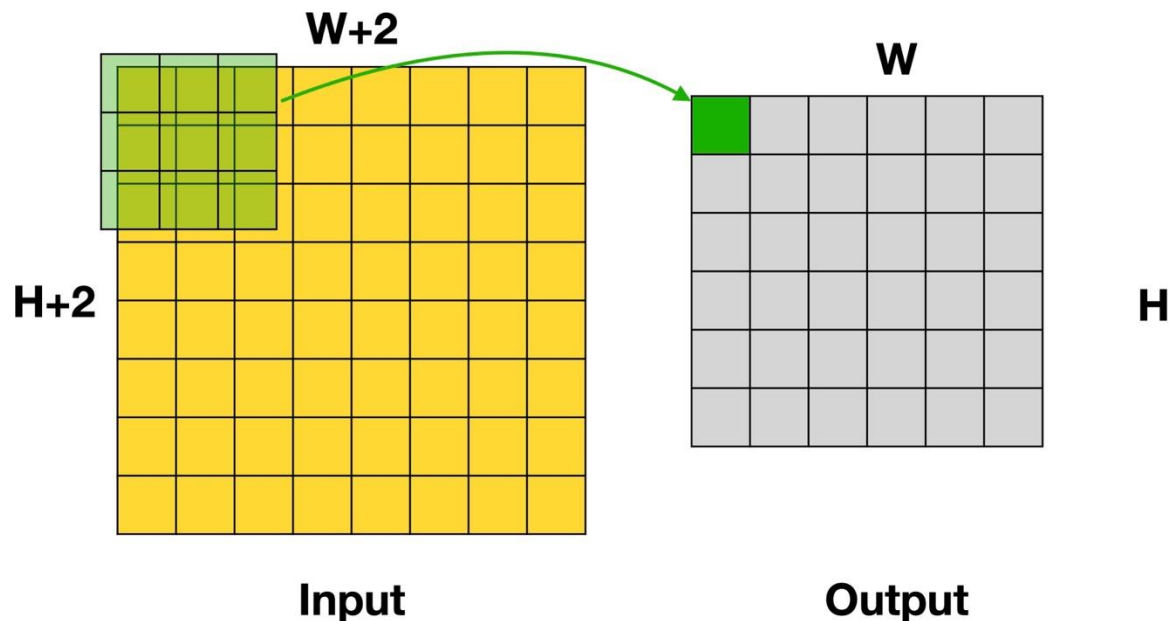
- AMD GPU 编程
 - CUDA → ROCm (开源)
 - API: `cudaXXXX()` → `hipXXXX()`
 - Kernel: `func<<< >>>()` ;
 - 计算库:
 - rocBLAS
 - rocRAND
 - RCCL
 - rocThrust

总结

- CUDA 编程模型
 - Nvidia GPU 上比较流行编程模型
 - 给 GPU 编程提供了一定的便利性
 - 提供 SPMD 编程风格
 - 但撰写高效 CUDA 程序仍需程序员显著参与
 - `kernelFunc <<< nB, nT, nS, Sid >>>` 创建一组线程
 - 线程块、线程组织

小作业-4

- 熟悉 GPU 编程
 - 简易版2维卷积 CUDA 程序
 - 测试和比较不同的线程块大小
 - 测试和比较使用 shared memory 带来优化效果
 - 同时考虑线程块设置和 shared memory 的使用，什么时候 shared memory 有效果，并思考原因



课后阅读

■ CUDA 编程手册

- <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>

■ Memory Coalescing

- [https://cvw.cac.cornell.edu/\(X\(1\)S\(fomrspncuiy0fzgtv5sdvszi\)\)/gpu/coalesced](https://cvw.cac.cornell.edu/(X(1)S(fomrspncuiy0fzgtv5sdvszi))/gpu/coalesced)

■ CUDA 性能实践

- <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>



Search docs

- ⊕ 1. Introduction
- ⊕ 2. Programming Model
- ⊕ 3. Programming Interface
- ⊕ 4. Hardware Implementation
- ⊕ 5. Performance Guidelines
 - 6. CUDA-Enabled GPUs
- ⊕ 7. C++ Language Extensions
- ⊕ 8. Cooperative Groups
- ⊕ 9. CUDA Dynamic Parallelism
- ⊕ 10. Virtual Memory Management
- ⊕ 11. Stream Ordered Memory Allocator
- ⊕ 12. Graph Memory Nodes
- ⊕ 13. Mathematical Functions
- ⊕ 14. C++ Language Support
- ⊕ 15. Texture Fetching
- ⊕ 16. Compute Capabilities
- ⊕ 17. Driver API
 - 18. CUDA Environment Variables
- ⊕ 19. Unified Memory Programming
- ⊕ 20. Lazy Loading
- ⊕ 21. Notices